

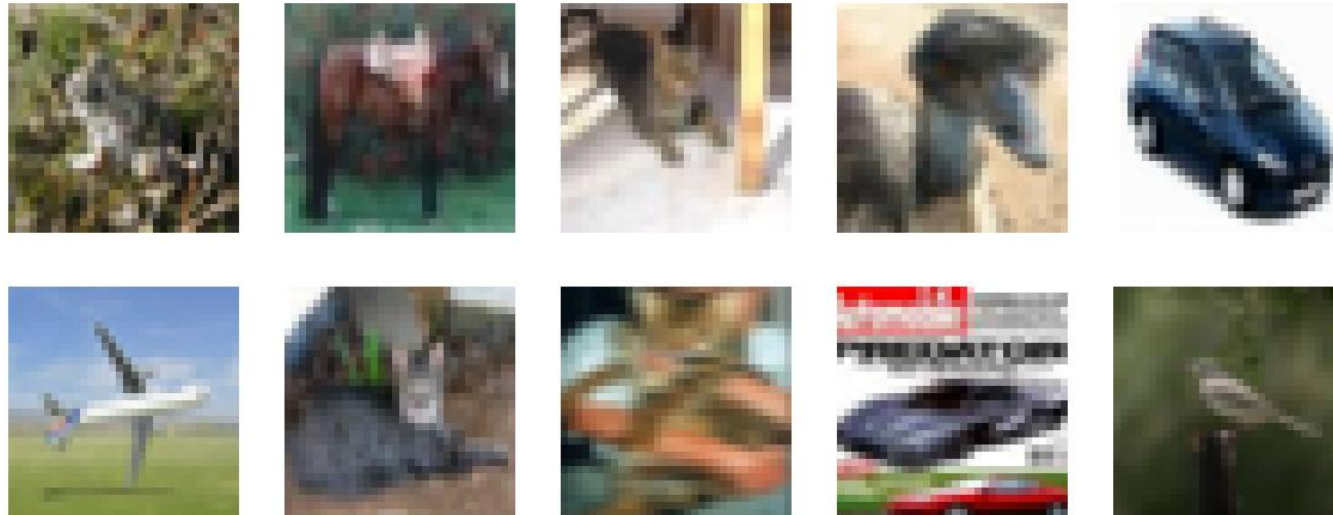
Epsilon Consistent Mixup:

Adaptive Data Augmentation in the Semi-Supervised Setting

Vincent Pisztor

A motivating example

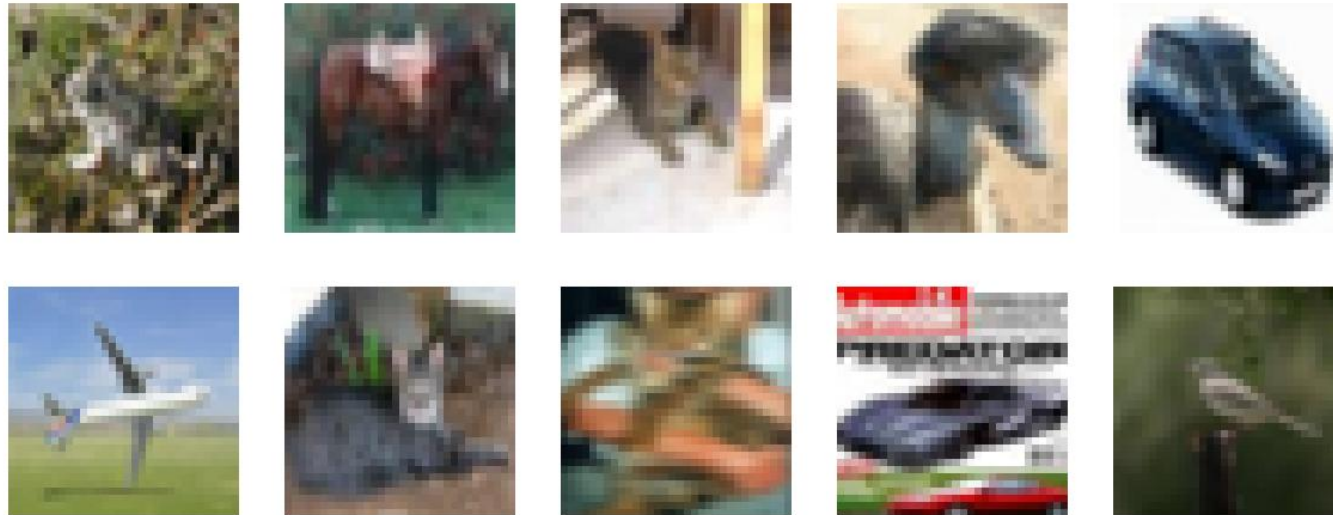
- Image classification with ten classes (CIFAR10)



- How accurate do you think you would be classifying these images?

A motivating example

- Image classification with ten classes (CIFAR10)



- How accurate do you think you would be classifying these images? 94.9%¹

¹Russakovsky, Olga et al. "ImageNet Large Scale Visual Recognition Challenge." International Journal of Computer Vision 115 (2015): 211-252.

A motivating example

- Image classification with ten classes (CIFAR10)
- Some more information:
 - 50,000 labeled examples (5,000 per class)
- How accurate do you think the best classifiers trained using these examples are?

A motivating example

- Image classification with ten classes (CIFAR10)
- Some more information:
 - 50,000 labeled examples (5,000 per class)
- How accurate do you think the best classifiers trained using these examples are? 99.4%¹

¹Kolesnikov, Alexander et al. “Big Transfer (BiT): General Visual Representation Learning.” ECCV (2020).

A motivating example

- Image classification with ten classes (CIFAR10)
- Some more information:
 - **250** labeled examples (**25** per class)
- How accurate do you think the best classifiers trained using these examples are?

A motivating example

- Image classification with ten classes (CIFAR10)
- Some more information:
 - **250** labeled examples (**25** per class)
- How accurate do you think the best classifiers trained using these examples are? 42.5%

A motivating example

- Image classification with ten classes (CIFAR10)
- Some more information:
 - **250** labeled examples (**25** per class)
- How accurate do you think the best classifiers trained using these examples are? 42.5%
- What if I told you that you could improve performance to 87.44% for free?

A motivating example

- Image classification with ten classes (CIFAR10)
- Some more information:
 - **250** labeled examples (**25** per class)
- How accurate do you think the best classifiers trained using these examples are? 42.5%
- What if I told you that you could improve performance to 87.44% for free?



A motivating example

- Image classification with ten classes (CIFAR10)
- Some more information:
 - 250 labeled examples (25 per class) and **49,750** unlabeled examples (**4,975** per class)
- How accurate do you think the best classifiers trained using these examples are? 42.5%
- What if I told you that you could improve performance to 87.44% for free?



Outline

- Semi-Supervised Learning
 - Motivation
 - Setting
 - Assumptions
- Structural Regularization
 - Consistency Regularization
 - Mixup Interpolation
- ϵ -Consistent Mixup
 - Methodology
 - Results

Semi-Supervised Learning

Motivation

Setting

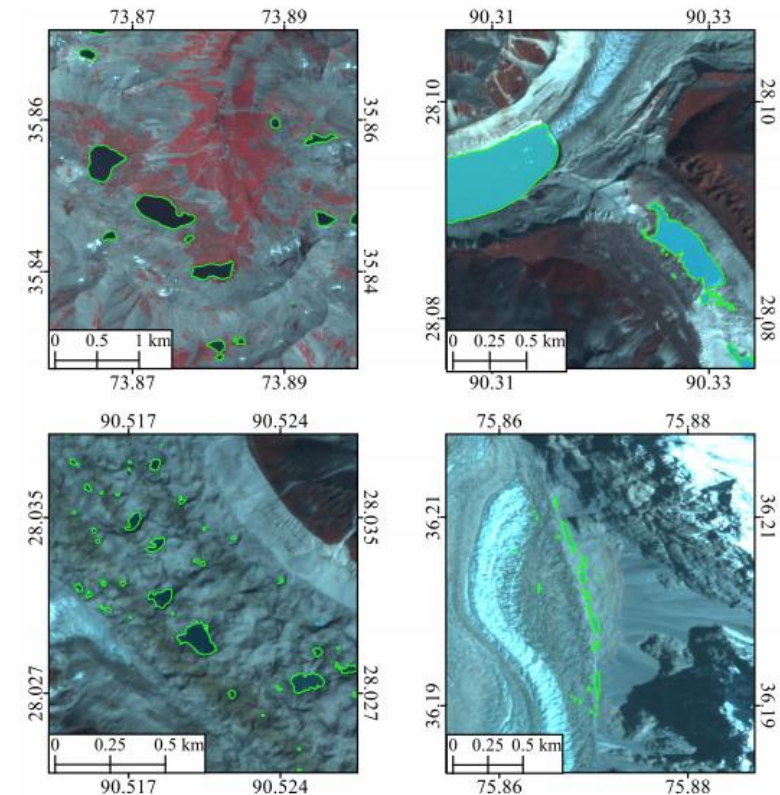
Assumptions

Motivation

- More performance with less labeled data
 - Lots of unlabeled observations
 - Few labeled observations

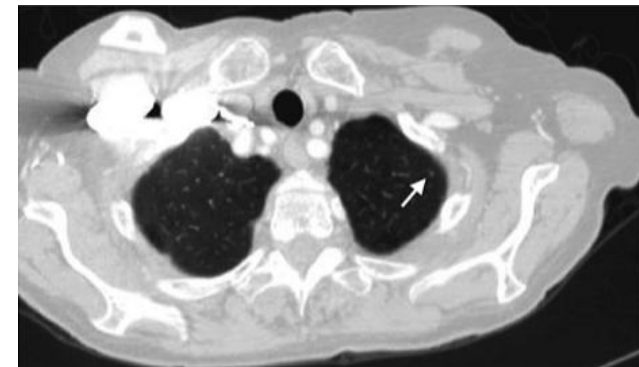
Motivation

- More performance with less labeled data
 - Lots of unlabeled observations
 - Few labeled observations
 - **Expensive to obtain**



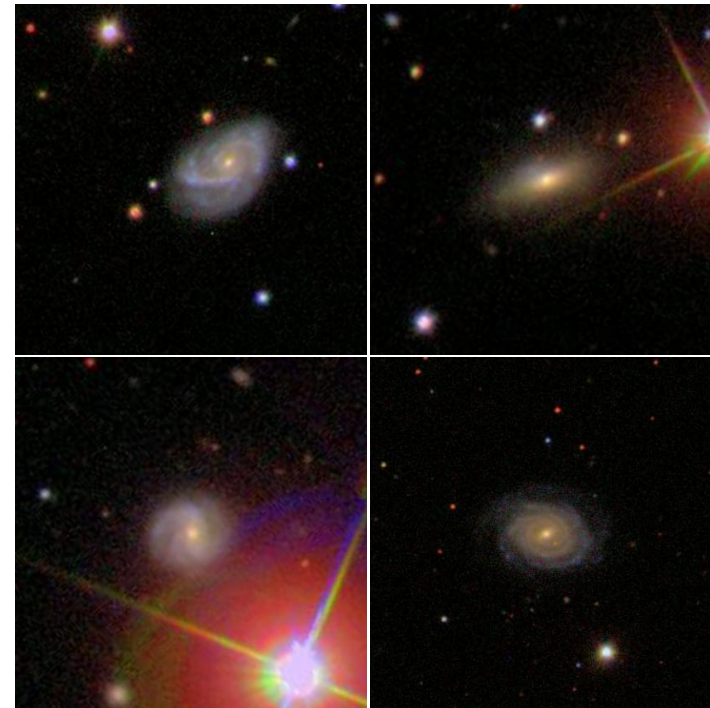
Motivation

- More performance with less labeled data
 - Lots of unlabeled observations
 - Few labeled observations
 - Expensive to obtain
 - **Expert annotation needed**



Motivation

- More performance with less labeled data
 - Lots of unlabeled observations
 - Few labeled observations
 - Expensive to obtain
 - Expert annotation needed
 - **Too much data to label**



Setting: Semi-Supervised

- Dataset: $D = \{D_L^{n_L}, D_U^{n_U}\}$
- Labeled Data: $D_L^{n_L} = \{(x_i, y_i): i = 1, \dots, n_L\}$
- Unlabeled Data: $D_U^{n_U} = \{(x_i, y_i): y_i = \emptyset \forall i, i = n_L + 1, \dots, N\}$

Setting: Semi-Supervised Image Classification

- Dataset: $D = \{D_L^{n_L}, D_U^{n_U}\}$
- Labeled Data: $D_L^{n_L} = \{(x_i, y_i): i = 1, \dots, n_L\}$
- Unlabeled Data: $D_U^{n_U} = \{(x_i, y_i): y_i = \emptyset \forall i, i = n_L + 1, \dots, N\}$
- Observations:
 - Features: $x \in \mathbb{R}^{h \times w \times c}$ (pixel values of each image)
 - Response: $y = p(l|x) \in [0,1]^C \cup \emptyset, \sum_{i=1}^C y_i = 1$ (distribution over C classes)
- Task: Classification
 - Estimate y with the neural network $f_\theta(x)$ parameterized by θ , using $D_L^{n_L}$ and $D_U^{n_U}$ for training.

Setting: Semi-Supervised Image Classification

- Dataset: $D = \{D_L^{n_L}, D_U^{n_U}\}$
- Labeled Data: $D_L^{n_L} = \{(x_i, y_i): i = 1, \dots, n_L\}$
- Unlabeled Data: $D_U^{n_U} = \{(x_i, y_i): y_i = \emptyset \ \forall i, i = n_L + 1, \dots, N\}$
- Observations:
 - Features: $x \in \mathbb{R}^{h \times w \times c}$ (pixel values of each image)
 - Response: $y = p(l|x) \in [0,1]^C \cup \emptyset, \sum_{i=1}^C y_i = 1$ (distribution over C classes)
- Task: Classification
 - Estimate y with the neural network $f_\theta(x)$ parameterized by θ , using $D_L^{n_L}$ and $D_U^{n_U}$ for training.

$$y = \begin{array}{|c|c|c|c|} \hline 0.7 & 0.0 & 0.1 & 0.2 \\ \hline \end{array}$$

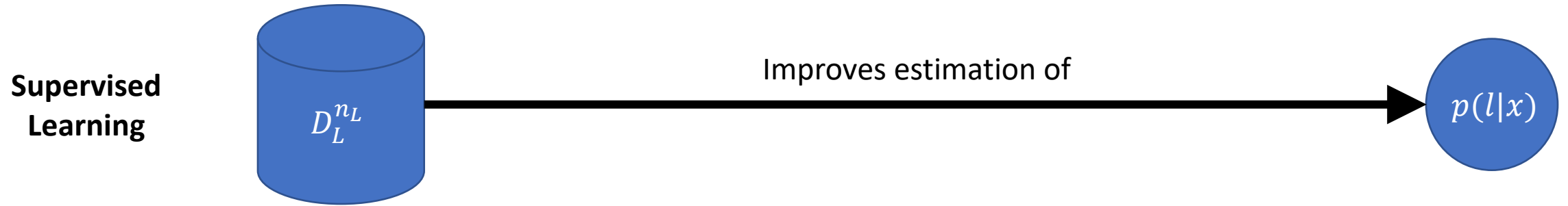
Setting: Semi-Supervised Image Classification

- Dataset: $D = \{D_L^{n_L}, D_U^{n_U}\}$
- Labeled Data: $D_L^{n_L} = \{(x_i, y_i) : i = 1, \dots, n_L\}$
- Unlabeled Data: $D_U^{n_U} = \{(x_i, y_i) : y_i = \emptyset \ \forall i, i = n_L + 1, \dots, N\}$
- Observations:
 - Features: $x \in \mathbb{R}^{h \times w \times c}$ (pixel values of each image)
 - Response: $y = p(l|x) \in [0,1]^C \cup \emptyset, \sum_{i=1}^C y_i = 1$ (distribution over C classes)
- Task: Classification
 - Estimate y with the neural network $f_\theta(x)$ parameterized by θ , using $D_L^{n_L}$ and $D_U^{n_U}$ for training.

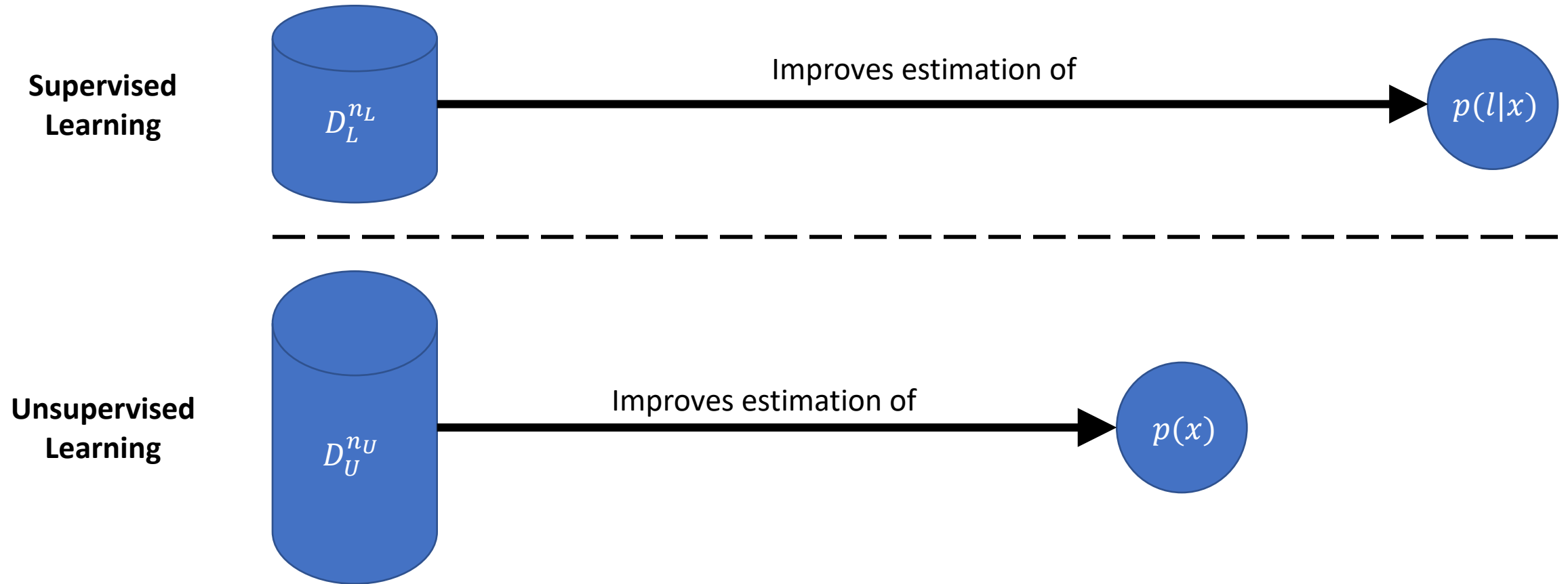
Setting: Semi-Supervised Image Classification

- Dataset: $D = \{D_L^{n_L}, D_U^{n_U}\}$
- Labeled Data: $D_L^{n_L} = \{(x_i, y_i) : i = 1, \dots, n_L\}$
- Unlabeled Data: $D_U^{n_U} = \{(x_i, y_i) : y_i = \emptyset \ \forall i, i = n_L + 1, \dots, N\}$
- Observations:
 - Features: $x \in \mathbb{R}^{h \times w \times c}$ (pixel values of each image)
 - Response: $y = p(l|x) \in [0,1]^C \cup \emptyset, \sum_{i=1}^C y_i = 1$ (distribution over C classes)
- Task: Classification
 - Estimate y with the neural network $f_\theta(x)$ parameterized by θ , **using $D_L^{n_L}$ and $D_U^{n_U}$ for training.**

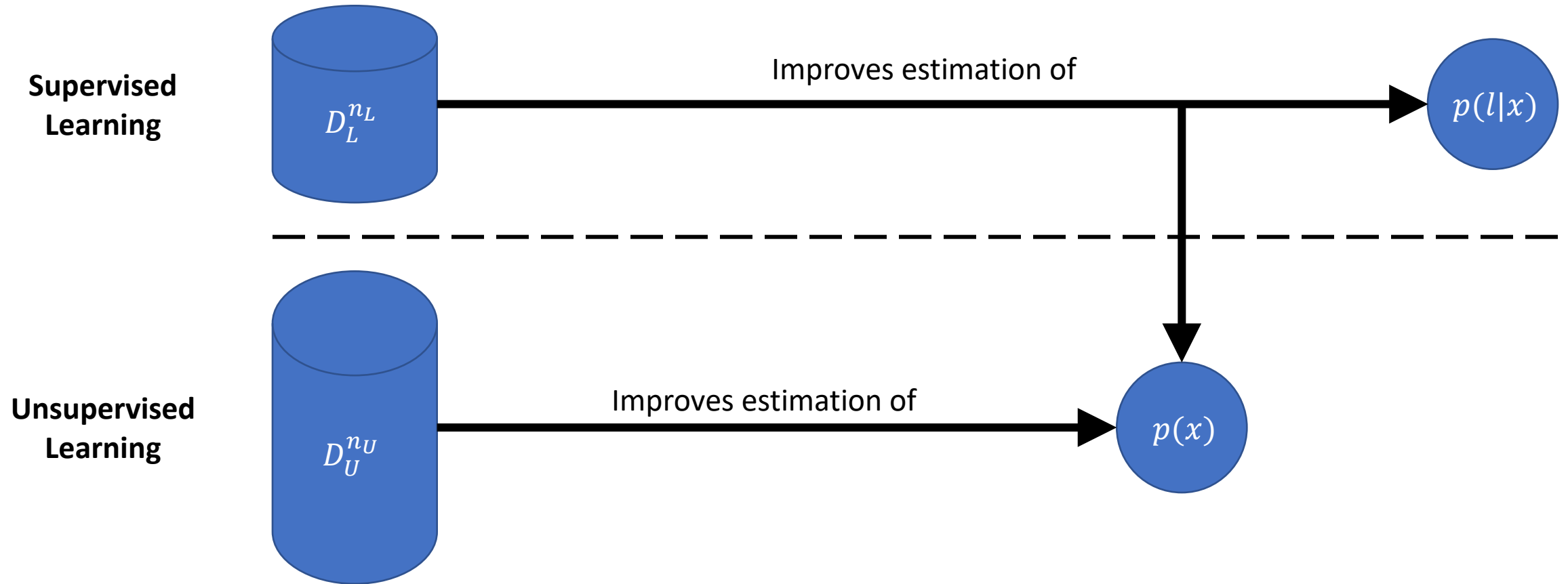
Semi-Supervised Learning: Intuition



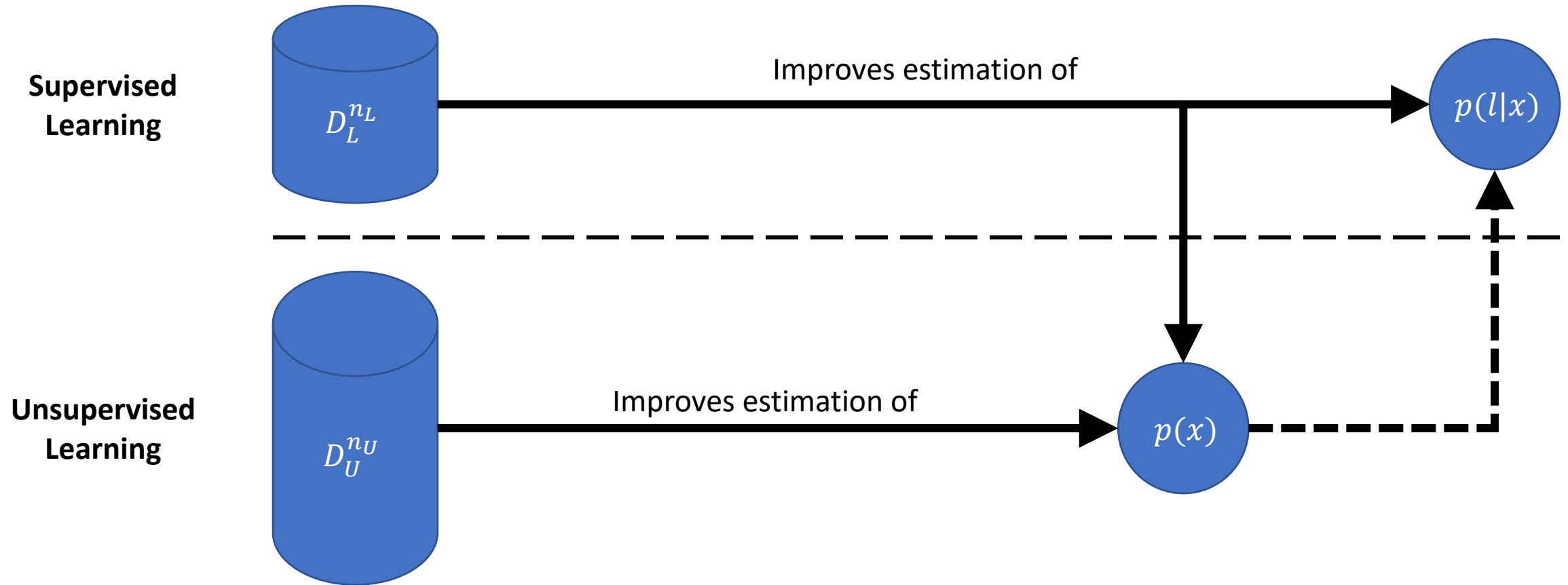
Semi-Supervised Learning: Intuition



Semi-Supervised Learning: Intuition



Semi-Supervised Learning: Intuition

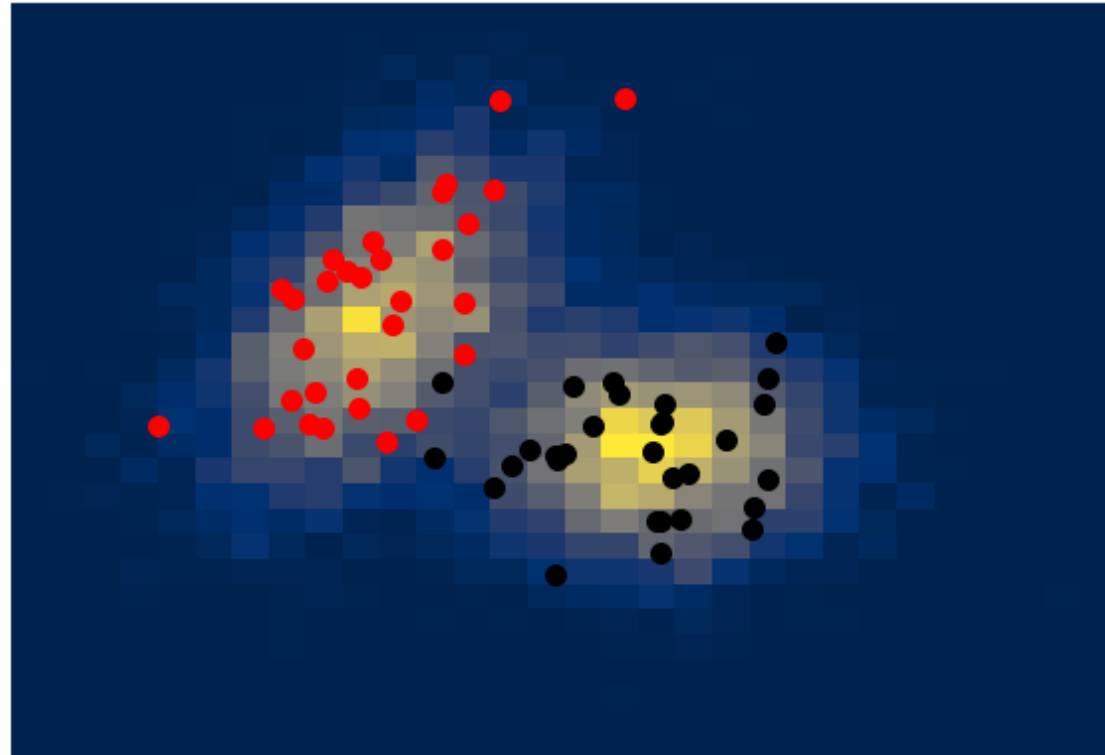


Assumptions

- Semi-supervised smoothness assumption: Proximity of observations i and j in a high-density region of the feature space implies proximity of their respective conditional distributions, $p(l|x_i)$ and $p(l|x_j)$.

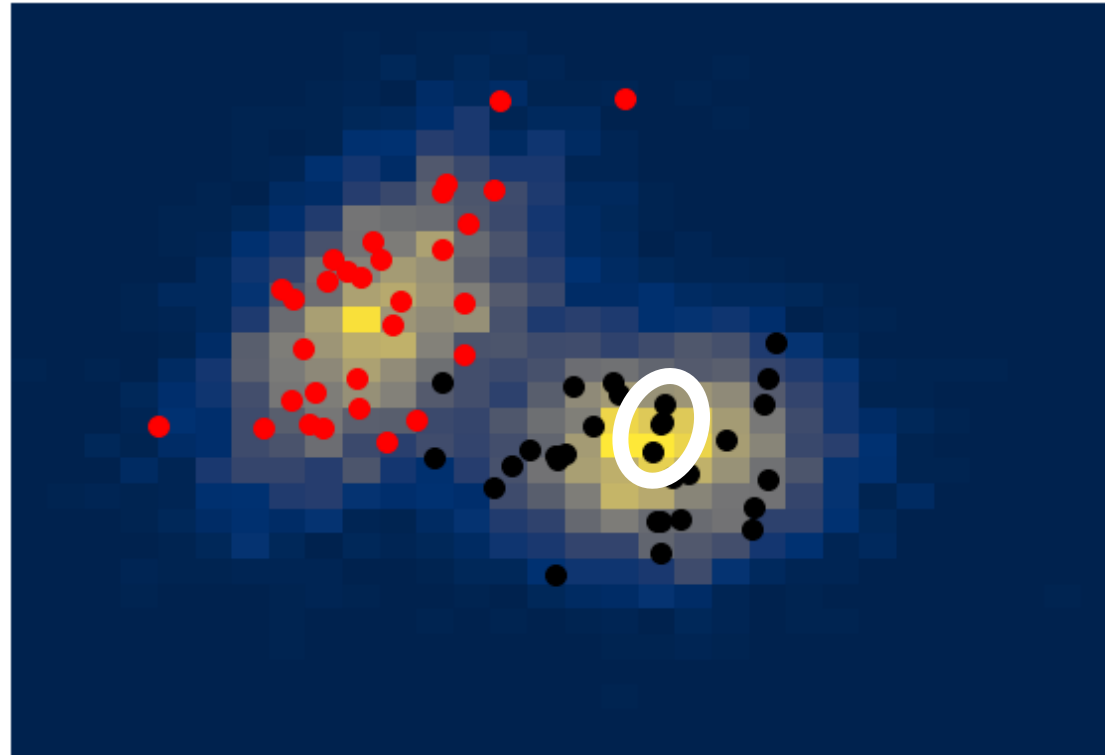
Assumptions

- Semi-supervised smoothness assumption: Proximity of observations i and j in a high-density region of the feature space implies proximity of their respective conditional distributions, $p(l|x_i)$ and $p(l|x_j)$.



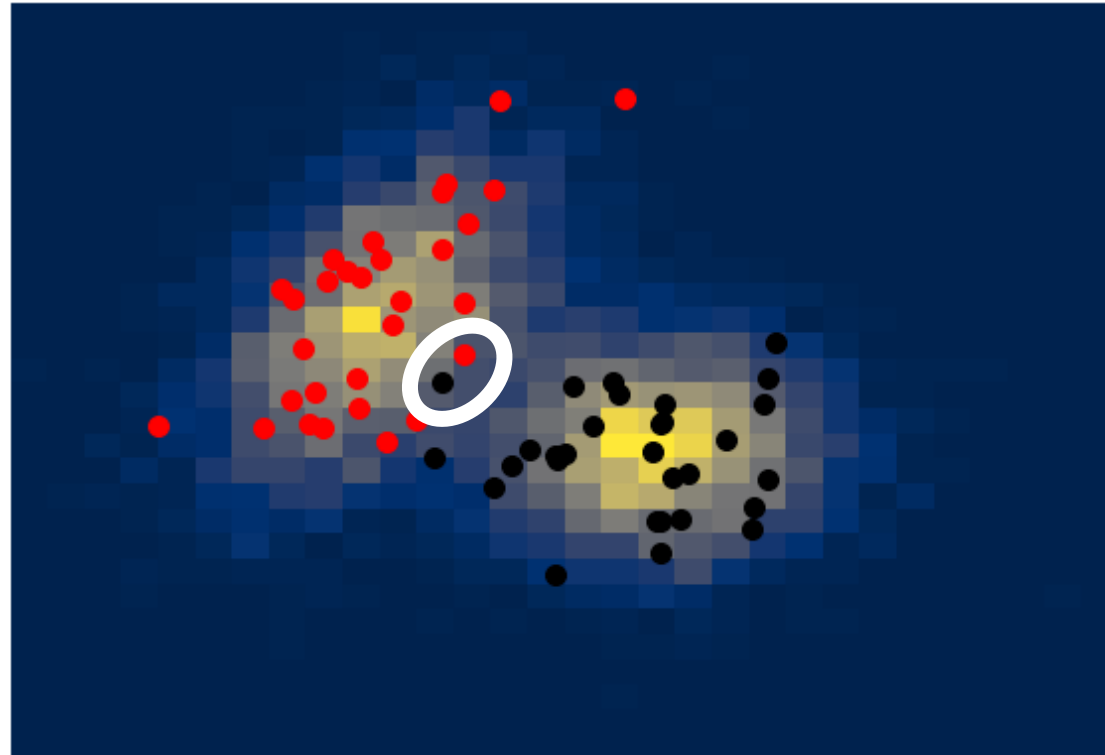
Assumptions

- Semi-supervised smoothness assumption: Proximity of observations i and j in a high-density region of the feature space implies proximity of their respective conditional distributions, $p(l|x_i)$ and $p(l|x_j)$.



Assumptions

- Semi-supervised smoothness assumption: Proximity of observations i and j in a high-density region of the feature space implies proximity of their respective conditional distributions, $p(l|x_i)$ and $p(l|x_j)$.

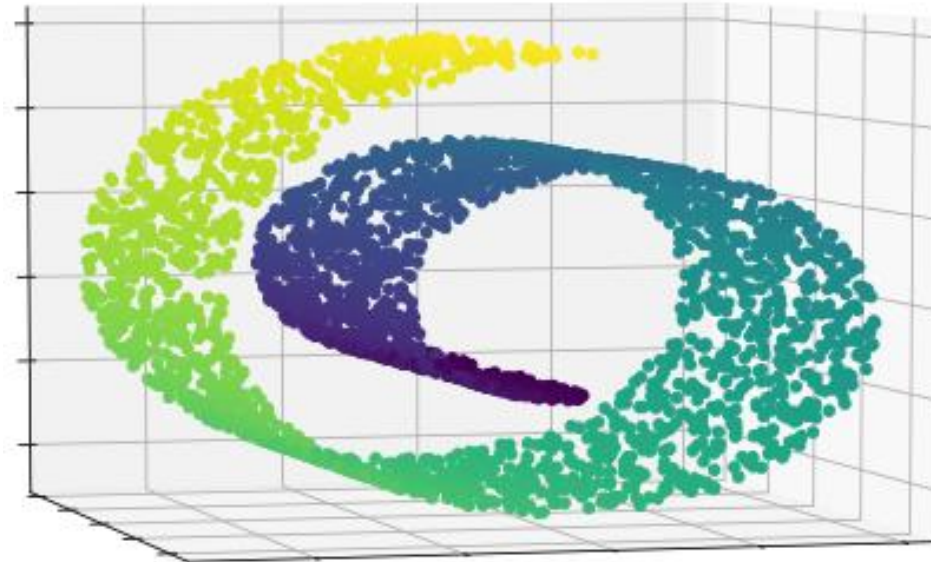


Assumptions

- Semi-supervised smoothness assumption: Proximity of observations i and j in a high-density region of the feature space implies proximity of their respective conditional distributions, $p(l|x_i)$ and $p(l|x_j)$.

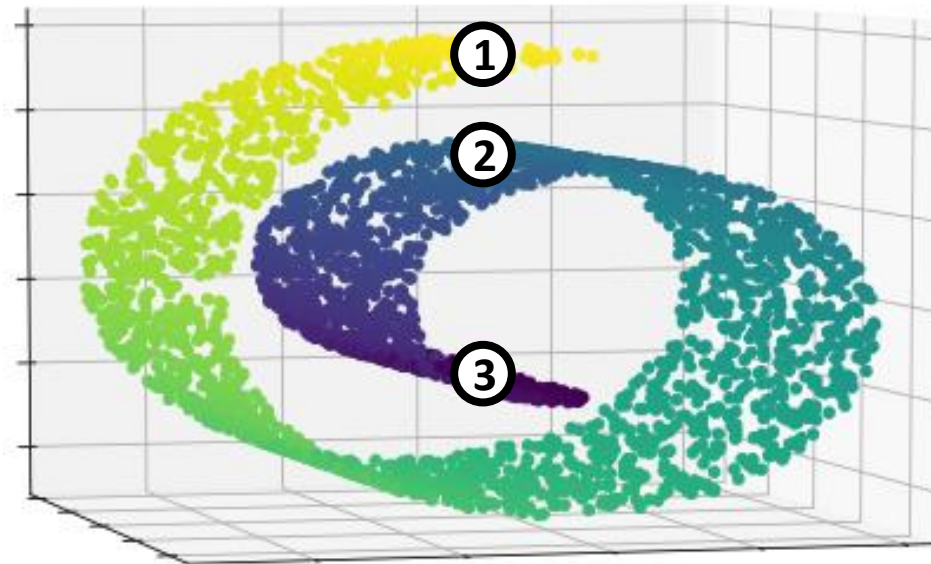
Assumptions

- Semi-supervised smoothness assumption: Proximity of observations i and j in a high-density region of the feature space implies proximity of their respective conditional distributions, $p(l|x_i)$ and $p(l|x_j)$.



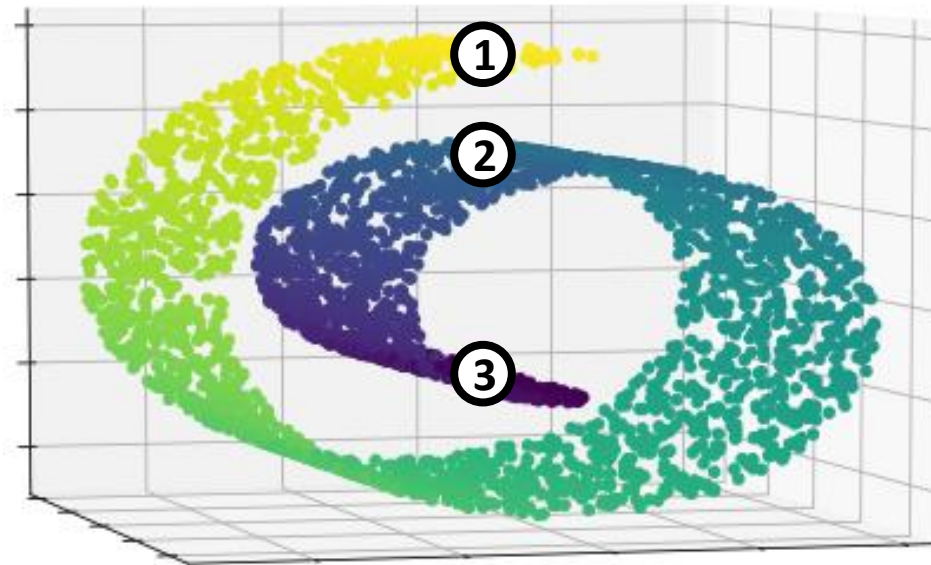
Assumptions

- Semi-supervised smoothness assumption: Proximity of observations i and j in a high-density region of the feature space implies proximity of their respective conditional distributions, $p(l|x_i)$ and $p(l|x_j)$.



Assumptions

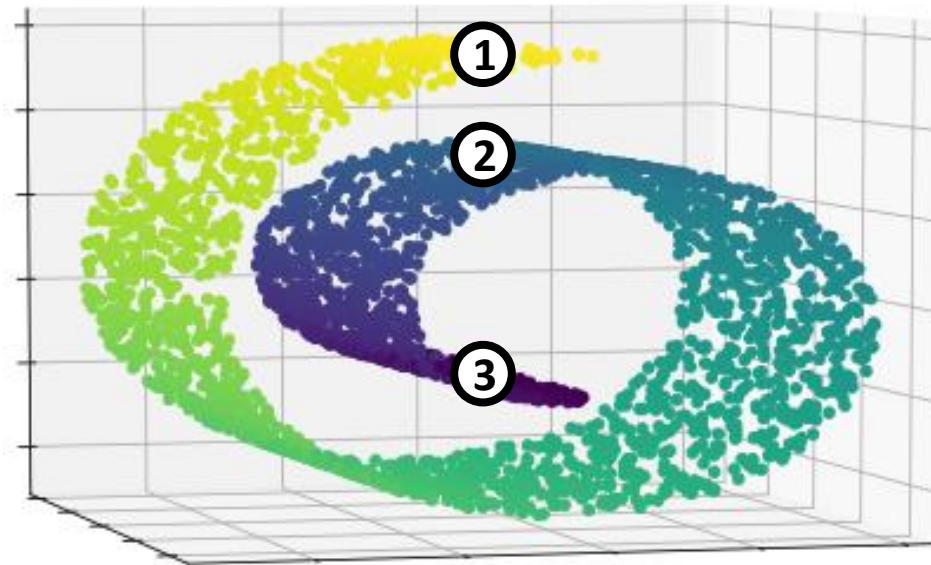
- Semi-supervised smoothness assumption: Proximity of observations i and j in a high-density region of the feature space implies proximity of their respective conditional distributions, $p(l|x_i)$ and $p(l|x_j)$.



$$D_F(\textcircled{1}, \textcircled{2}) < D_F(\textcircled{2}, \textcircled{3})$$

Assumptions

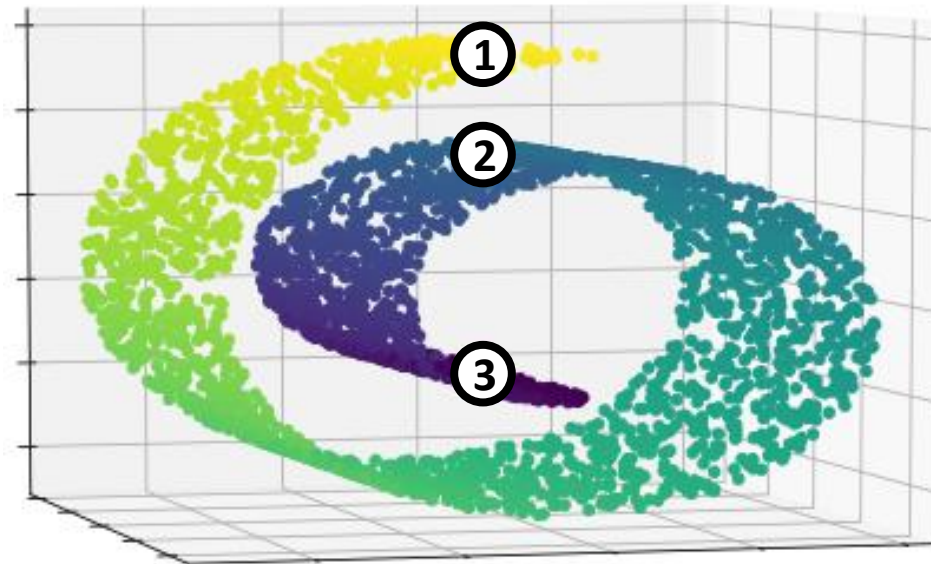
- Semi-supervised smoothness assumption: Proximity of observations i and j in a high-density region of the feature space implies proximity of their respective conditional distributions, $p(l|x_i)$ and $p(l|x_j)$.
- Manifold assumption: The data lie approximately on several low dimensional manifolds embedded in the feature space.



$$D_F(\textcircled{1}, \textcircled{2}) < D_F(\textcircled{2}, \textcircled{3})$$

Assumptions

- Semi-supervised smoothness assumption: Proximity of observations i and j in a high-density region of the feature space implies proximity of their respective conditional distributions, $p(l|x_i)$ and $p(l|x_j)$.
- Manifold assumption: The data lie approximately on several low dimensional manifolds embedded in the feature space.

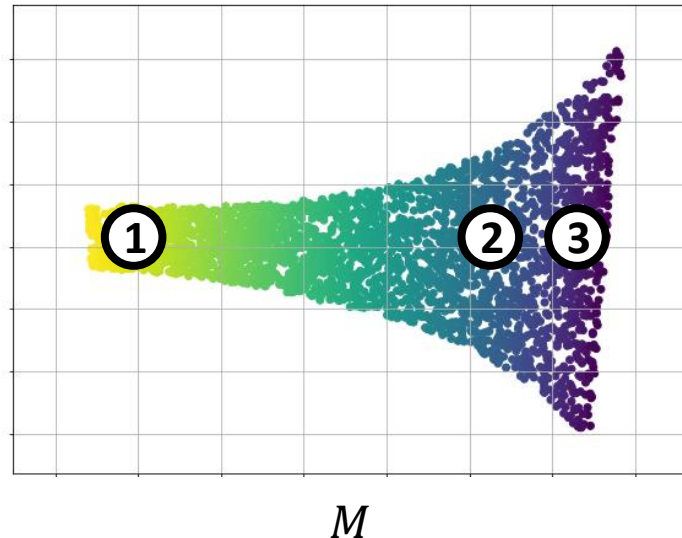
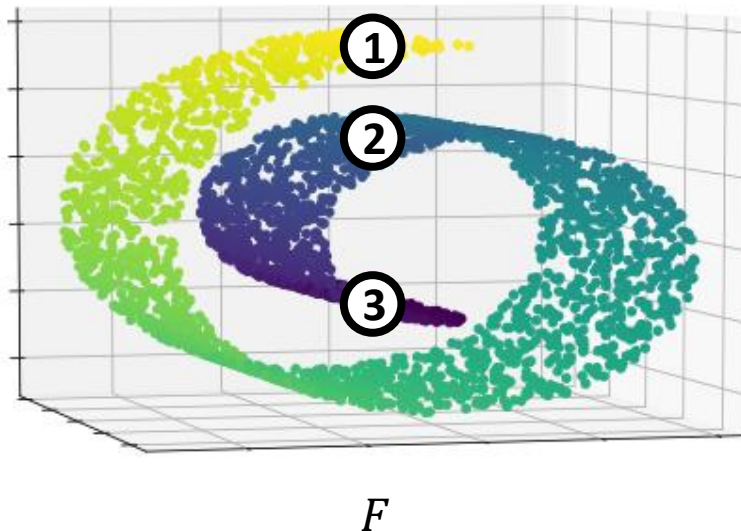


$$D_F(\textcircled{1}, \textcircled{2}) < D_F(\textcircled{2}, \textcircled{3})$$

$$D_M(\textcircled{1}, \textcircled{2}) > D_M(\textcircled{2}, \textcircled{3})$$

Assumptions

- Semi-supervised smoothness assumption: Proximity of observations i and j in a high-density region of the feature space implies proximity of their respective conditional distributions, $p(l|x_i)$ and $p(l|x_j)$.
- Manifold assumption: The data lie approximately on several low dimensional manifolds embedded in the feature space.



$$D_F(\textcircled{1}, \textcircled{2}) < D_F(\textcircled{2}, \textcircled{3})$$

$$D_M(\textcircled{1}, \textcircled{2}) > D_M(\textcircled{2}, \textcircled{3})$$

Structural Regularization

Consistency Regularization

Mixup Interpolation

Structural Regularization: Intuition

$$\hat{f} = \operatorname{argmin}_{f \in \mathcal{F}} L(f, D)$$

- Several ways to influence the model fitting process to incorporate assumptions

Structural Regularization: Intuition

$$\hat{f} = \operatorname{argmin}_{f \in \mathcal{F}} L(f, D)$$

- Several ways to influence the model fitting process to incorporate assumptions
 1. Change the loss function to penalize f which do not conform to the assumptions

Structural Regularization: Intuition

$$\hat{f} = \operatorname{argmin}_{f \in \mathcal{F}} L(f, D)$$

- Several ways to influence the model fitting process to incorporate assumptions
 1. Change the loss function to penalize f which do not conform to the assumptions
 2. Restrict $\mathcal{F}$ to functions which conform to the assumptions

Structural Regularization: Intuition

$$\hat{f} = \operatorname{argmin}_{f \in \mathcal{F}} L(f, D)$$

- Several ways to influence the model fitting process to incorporate assumptions
 1. Change the loss function to penalize f which do not conform to the assumptions
 2. Restrict $\mathcal{F}$ to functions which conform to the assumptions
 3. Use a dataset D for training which conforms to the assumptions

Structural Regularization: Intuition

$$\hat{f} = \operatorname{argmin}_{f \in \mathcal{F}} L(f, D)$$

- Several ways to influence the model fitting process to incorporate assumptions
 1. Change the loss function to penalize f which do not conform to the assumptions
 2. Restrict $\mathcal{F}$ to functions which conform to the assumptions
 3. Use a dataset D for training which conforms to the assumptions
- Idea: Generate a synthetic dataset $\tilde{D}$ which obeys desired assumptions, then use this synthetic dataset to train a model

Structural Regularization: Intuition

$$\hat{f} = \operatorname{argmin}_{f \in \mathcal{F}} L(f, \tilde{D})$$

- Several ways to influence the model fitting process to incorporate assumptions
 1. Change the loss function to penalize f which do not conform to the assumptions
 2. Restrict $\mathcal{F}$ to functions which conform to the assumptions
 3. Use a dataset D for training which conforms to the assumptions
- Idea: Generate a synthetic dataset $\tilde{D}$ which obeys desired assumptions, then use this synthetic dataset to train a model

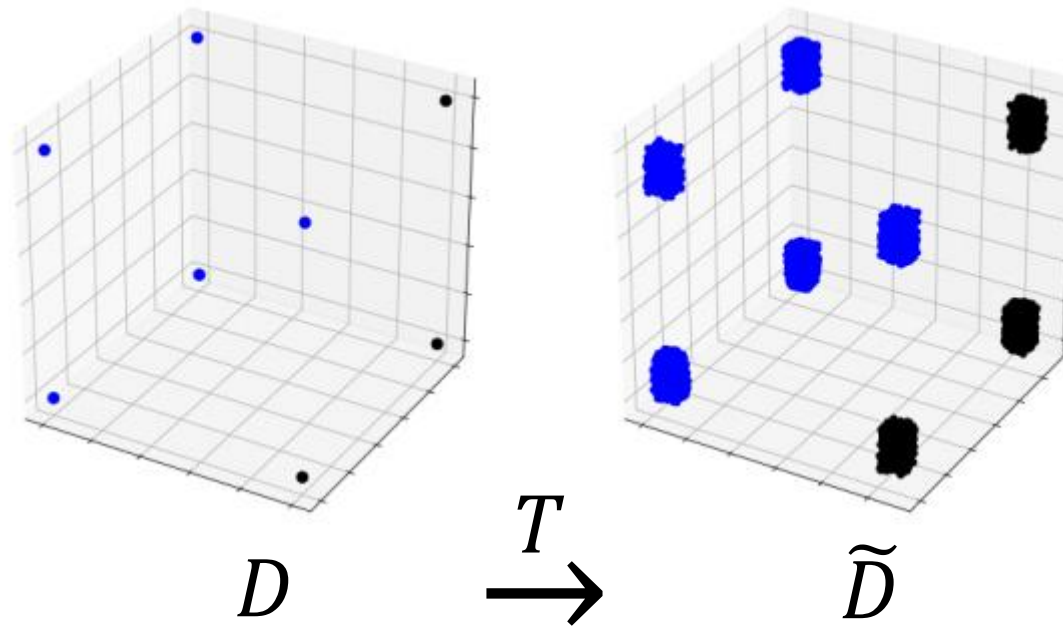
Structural Regularization: Intuition

$$\hat{f} = \operatorname{argmin}_{f \in \mathcal{F}} L(f, D)$$

- Several ways to influence the model fitting process to incorporate assumptions
 1. Change the loss function to penalize f which do not conform to the assumptions
 2. Restrict $\mathcal{F}$ to functions which conform to the assumptions
 3. Use a dataset D for training which conforms to the assumptions
- Idea: Generate a synthetic dataset $\tilde{D}$ which obeys desired assumptions, then use this synthetic dataset to train a model
- Why? Training using $\tilde{D}$ will encourage selection of a model that conforms to the assumptions built into the data

Structural Regularization: Intuition

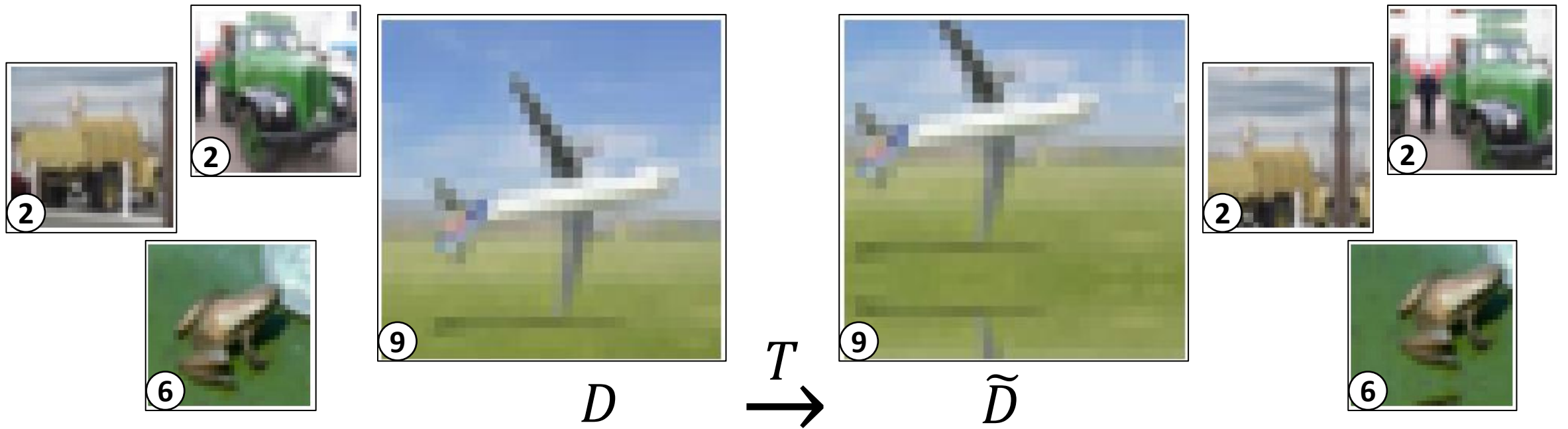
- Example: Local class consistency



- Training using $\tilde{D}$ will produce classifiers that maintain label consistency locally around each point in D .

Structural Regularization: Intuition

- Example: Shift invariance



- Training using $\tilde{D}$ will produce classifiers that are invariant to shifts when assigning a label.

Structural Regularization: Benefits

- Straightforward imposition of intuitive assumptions (e.g. shift invariance) which would be difficult to specify directly

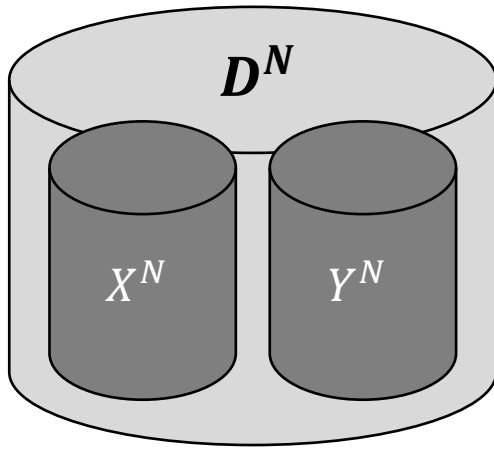
Structural Regularization: Benefits

- Straightforward imposition of intuitive assumptions (e.g. shift invariance) which would be difficult to specify directly
- Model agnostic regularization procedure

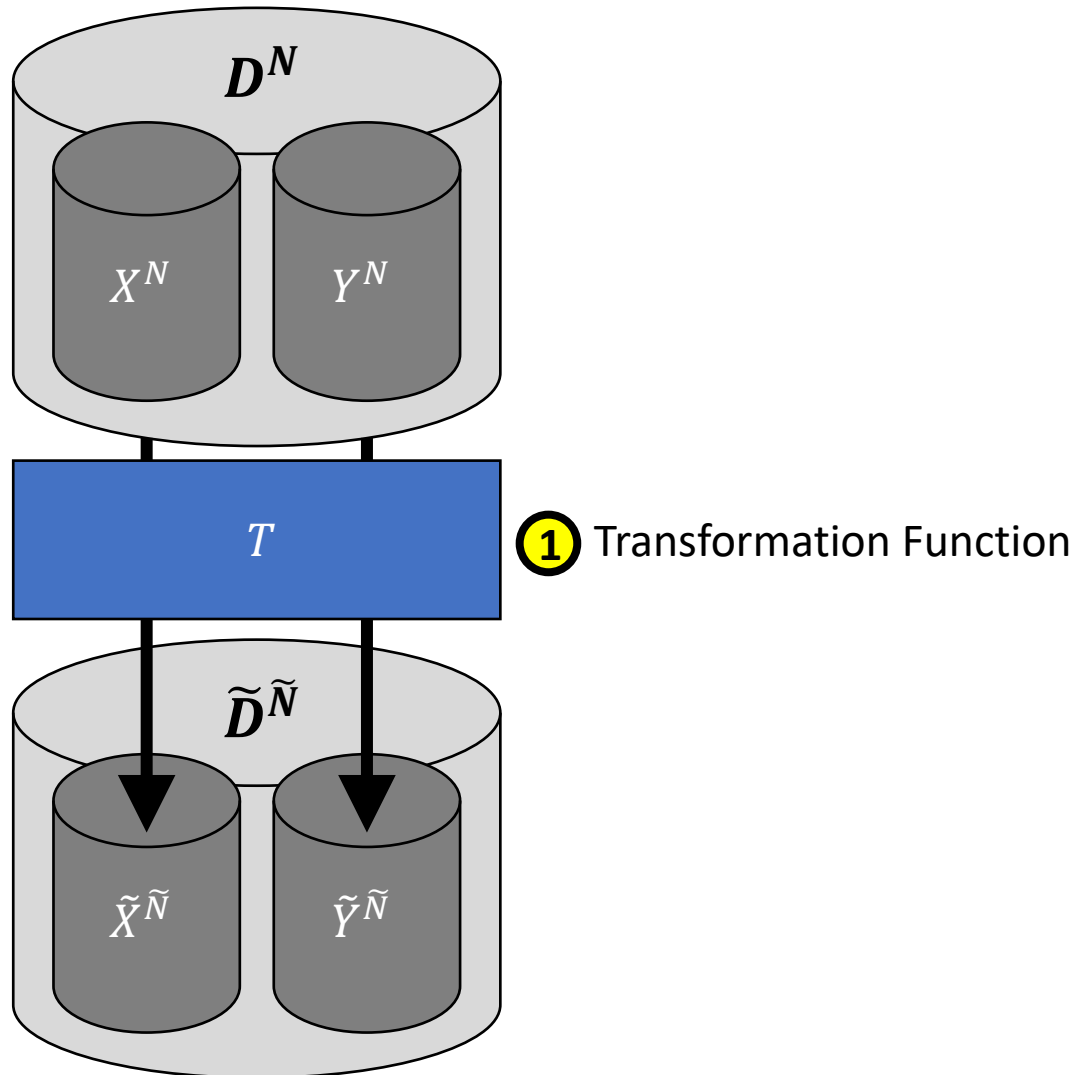
Structural Regularization: Benefits

- Straightforward imposition of intuitive assumptions (e.g. shift invariance) which would be difficult to specify directly
- Model agnostic regularization procedure
- Natural extension to semi-supervised settings

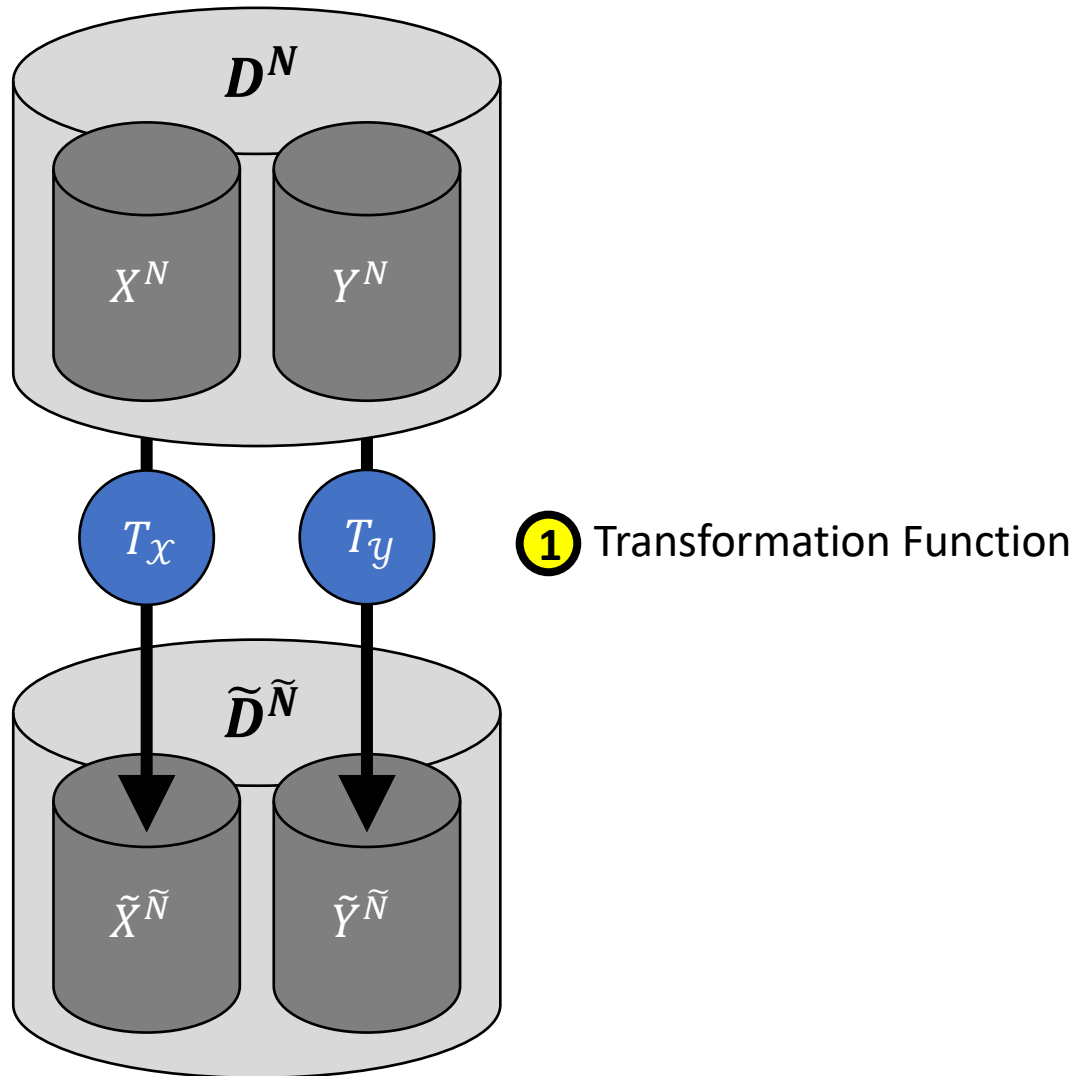
Structural Regularization: In General



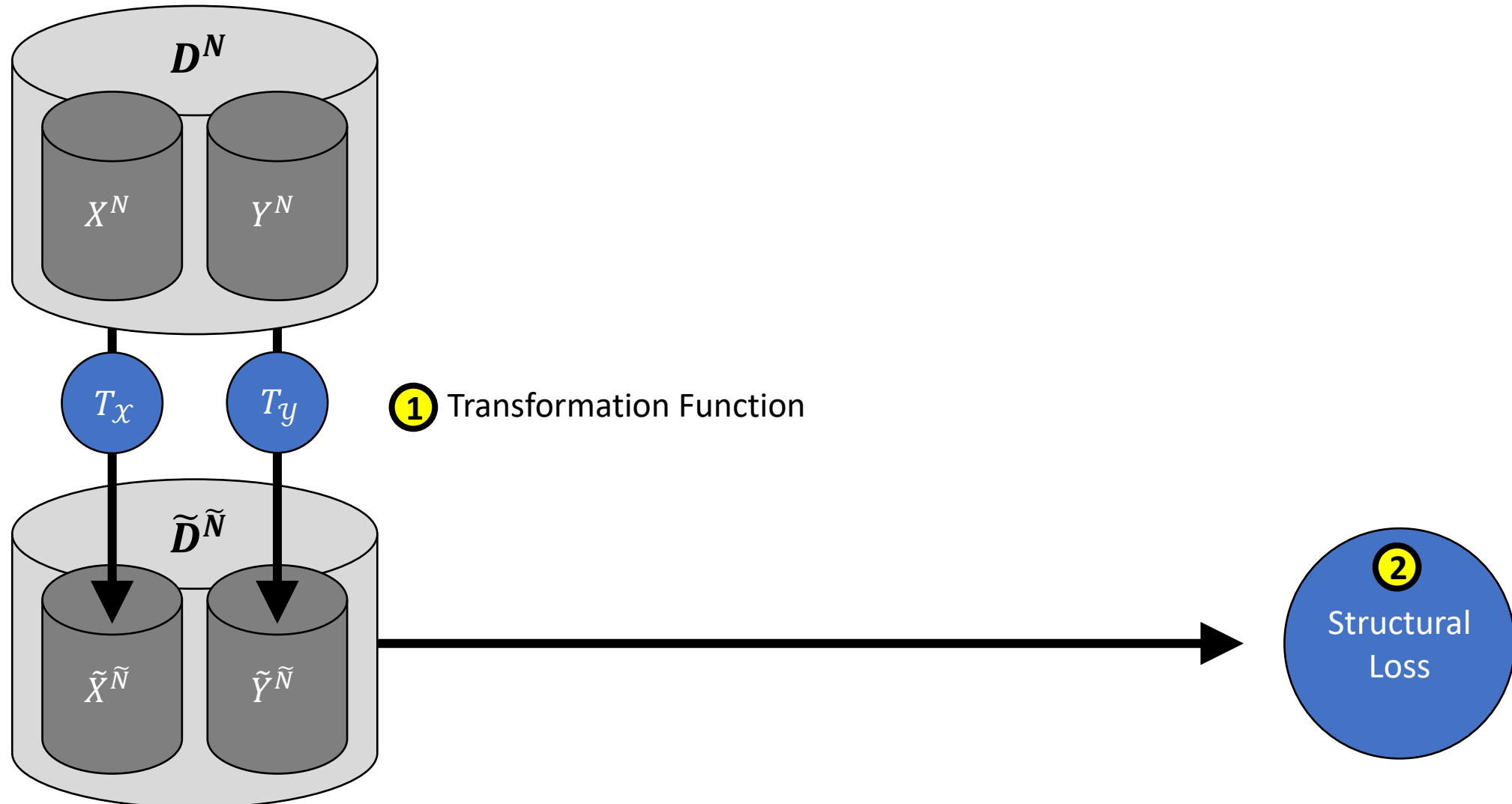
Structural Regularization: In General



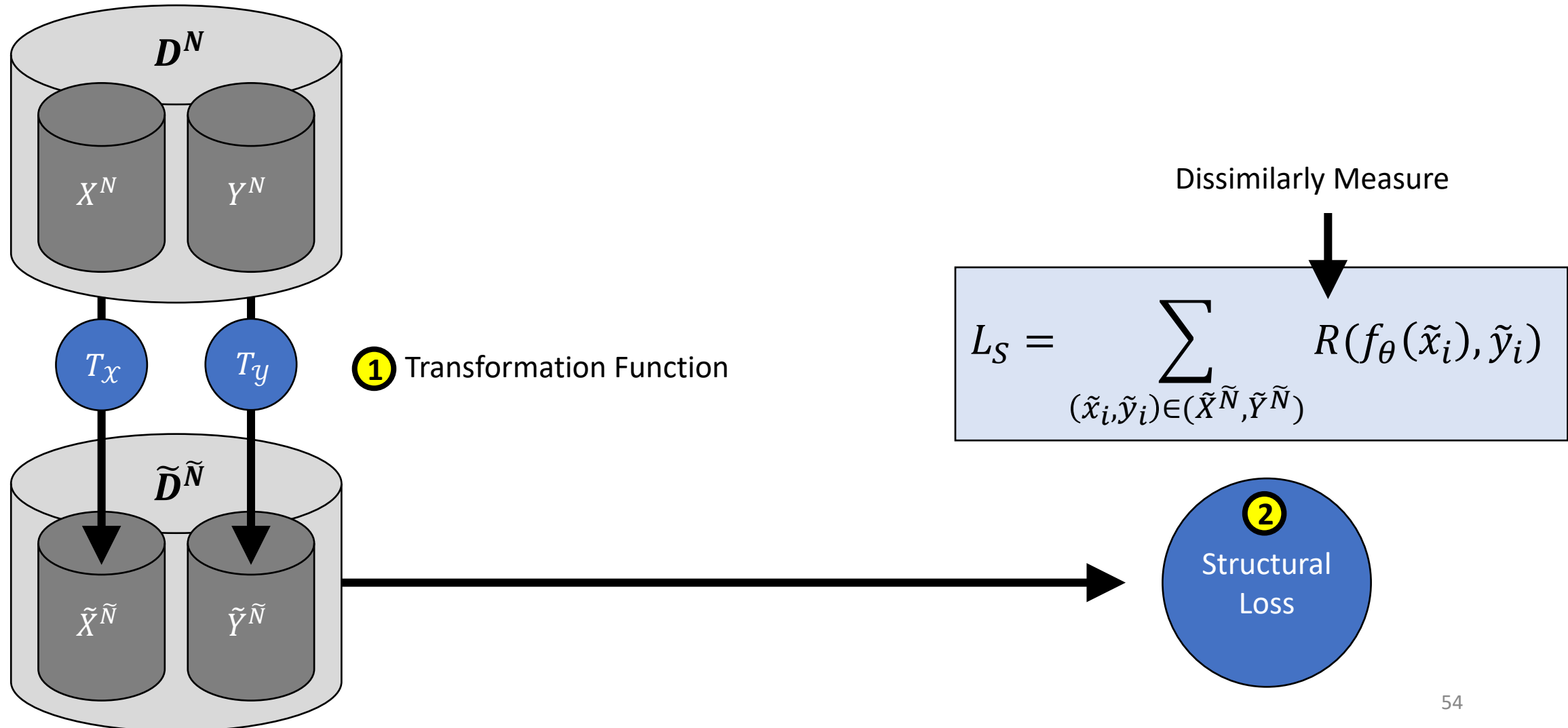
Structural Regularization: In General



Structural Regularization: In General



Structural Regularization: In General

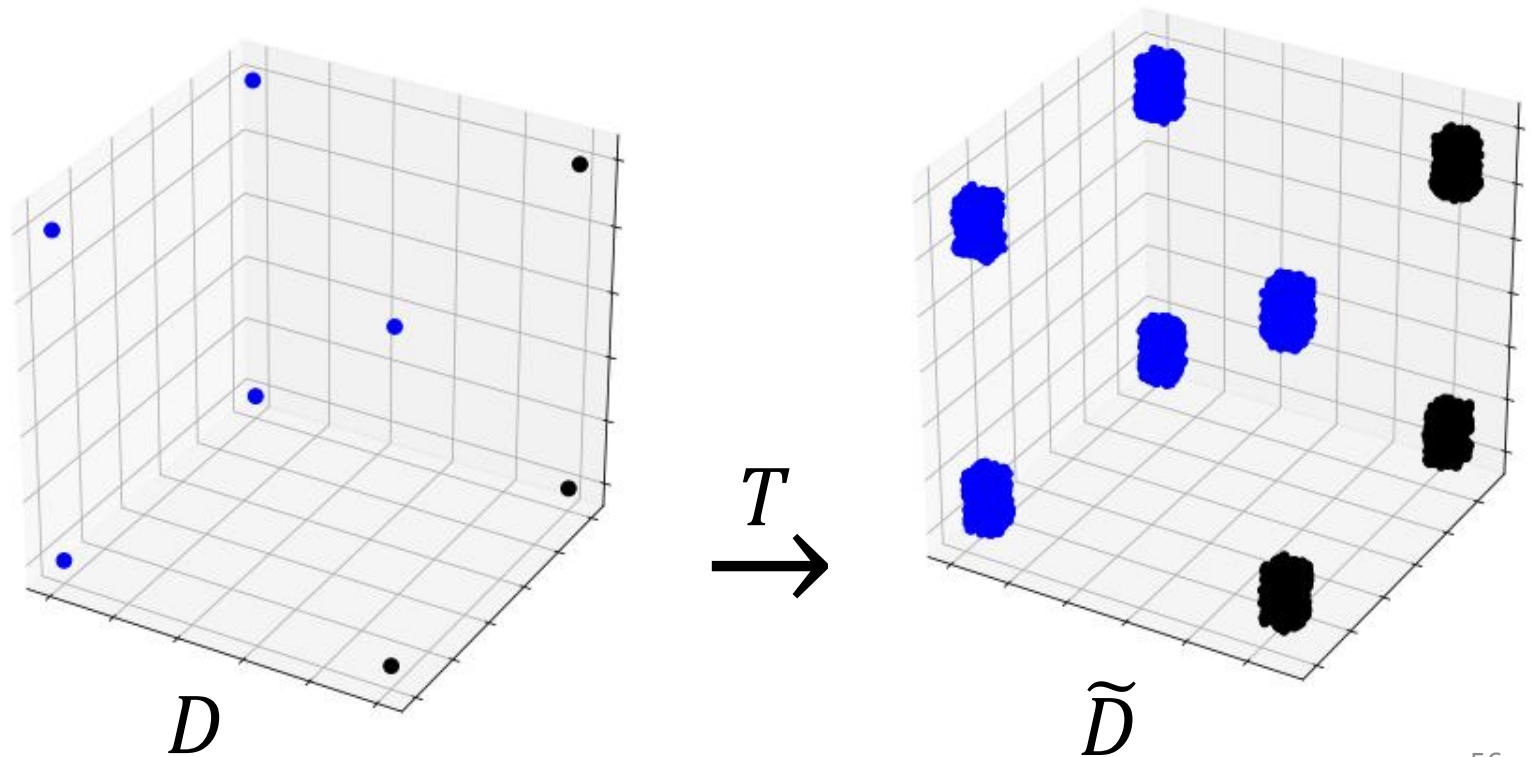


Structural Regularization: Consistency

- Idea: Generate many versions of each original observation and assign the original observation's label to each of the new synthetic observations.

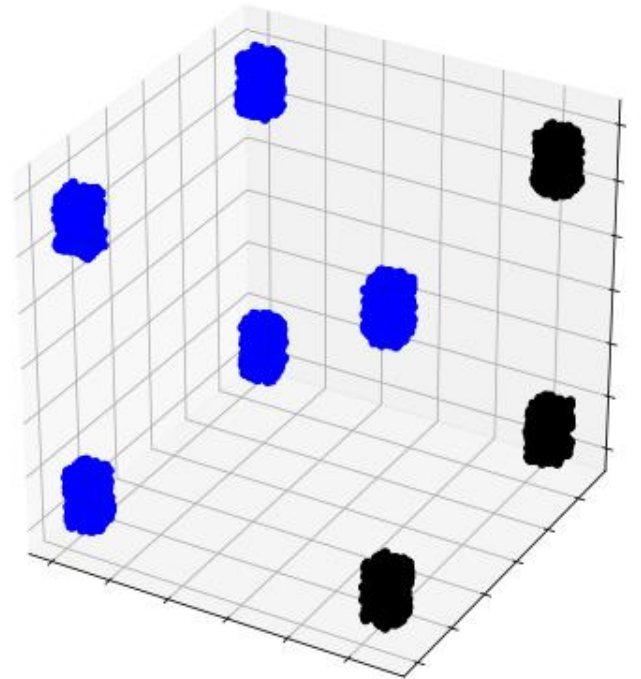
Structural Regularization: Consistency

- Idea: Generate many versions of each original observation and assign the original observation's label to each of the new synthetic observations.



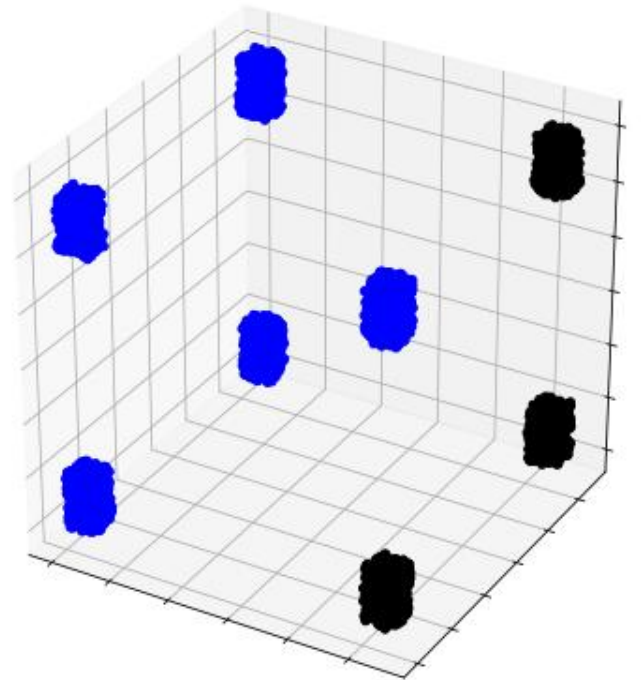
Structural Regularization: Consistency

- Idea: Generate many versions of each original observation and assign the original observation's label to each of the new synthetic observations.
- Transformation:
 - $T_{\mathcal{X}}$: Any class-preserving feature transformation
 - $T_{\mathcal{Y}}(y) = p(y) = \begin{cases} y, & y \neq \emptyset \\ \hat{p}, & y = \emptyset \end{cases} = \tilde{y}$



Structural Regularization: Consistency

- Idea: Generate many versions of each original observation and assign the original observation's label to each of the new synthetic observations.
- Transformation:
 - $T_{\mathcal{X}}$: Any class-preserving feature transformation
 - $T_{\mathcal{Y}}(y) = p(y) = \begin{cases} y, & y \neq \emptyset \\ \hat{p}, & y = \emptyset \end{cases} = \tilde{y}$
- Loss: MSE, KL-divergence, cross-entropy, etc.

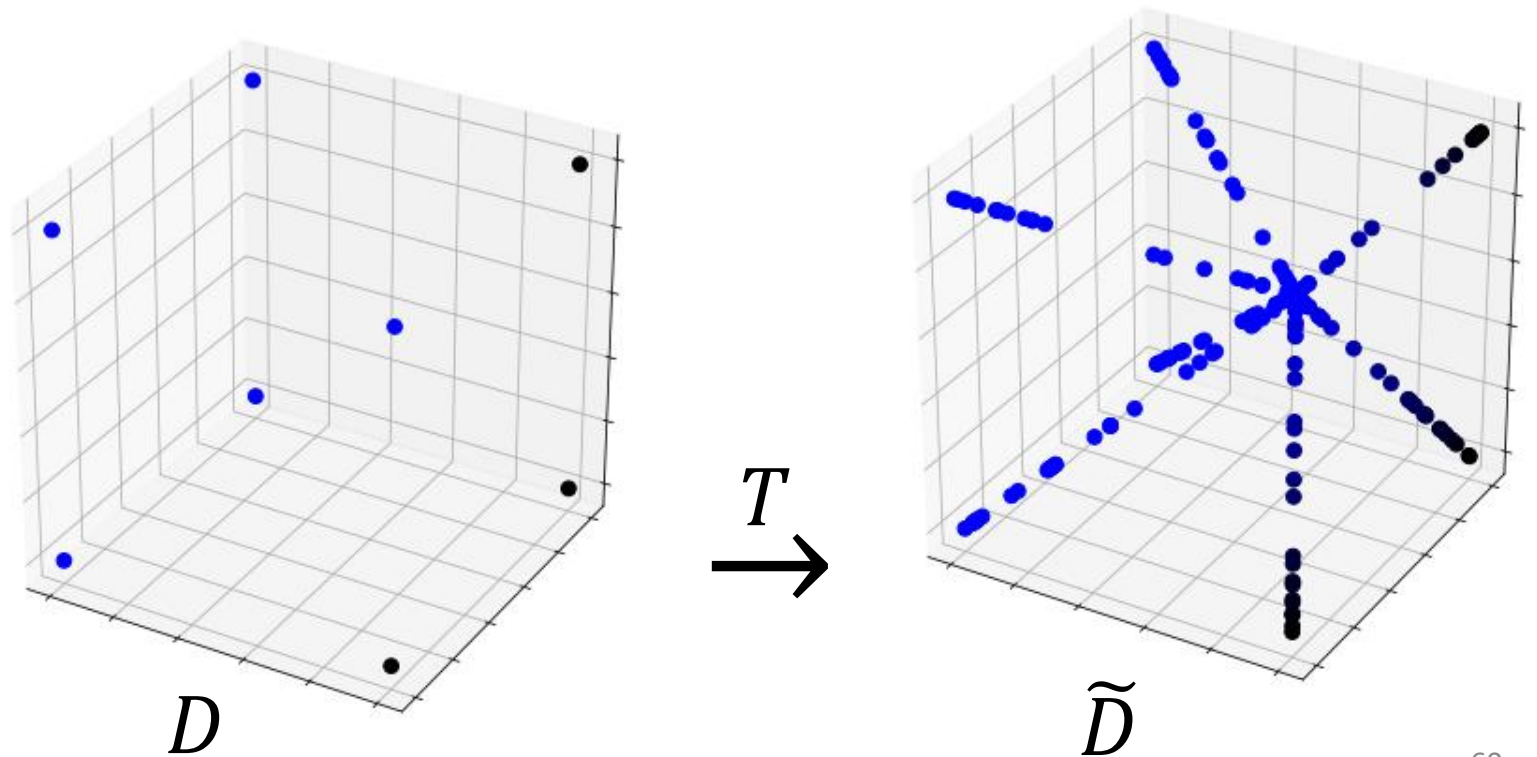


Structural Regularization: Interpolation (Mixup)

- Idea: Create a simple (interpolative) structure for the evolution of the response between pairs of observations.

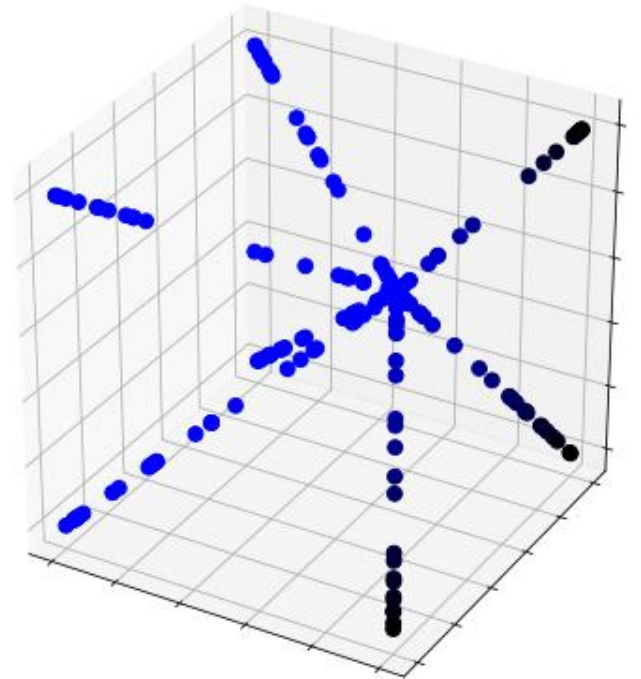
Structural Regularization: Interpolation (Mixup)

- Idea: Create a simple (interpolative) structure for the evolution of the response between pairs of observations.



Structural Regularization: Interpolation (Mixup)

- Idea: Create a simple (interpolative) structure for the evolution of the response between pairs of observations.
- Transformation:
 - $T_x(x_i, x_j) = \lambda x_i + (1 - \lambda)x_j, \lambda \sim \text{Beta}(\beta, \beta)$
 - $T_y(y_i, y_j) = \lambda p(y_i) + (1 - \lambda)p(x_j)$
 - $p(y) = \begin{cases} y, y \neq \emptyset \\ \hat{p}, y = \emptyset \end{cases}$
- Loss: MSE, KL-divergence, cross-entropy, etc.



Structural Regularization: The Semi-Supervised Piece

$$p(y) = \begin{cases} y, & y \neq \emptyset \\ \hat{p}, & y = \emptyset \end{cases}$$

- Intuition: $\hat{p} = f_{\theta}(x)$
- Replacing unknown y with a $\hat{p}$ that does not conflict with the model's predictions enables enforcement of the desired structure without otherwise interfering with the model's decision making.
- E.g. consistency is achieved between x and $\tilde{x}$ (by assigning $\hat{p}$ to both), without making any assumptions about the true label.

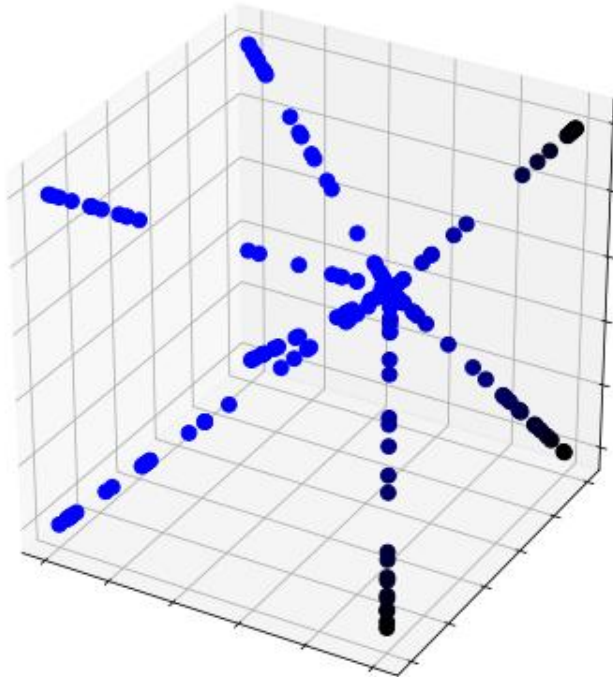
ϵ -Consistent Mixup (ϵmu)

Methodology

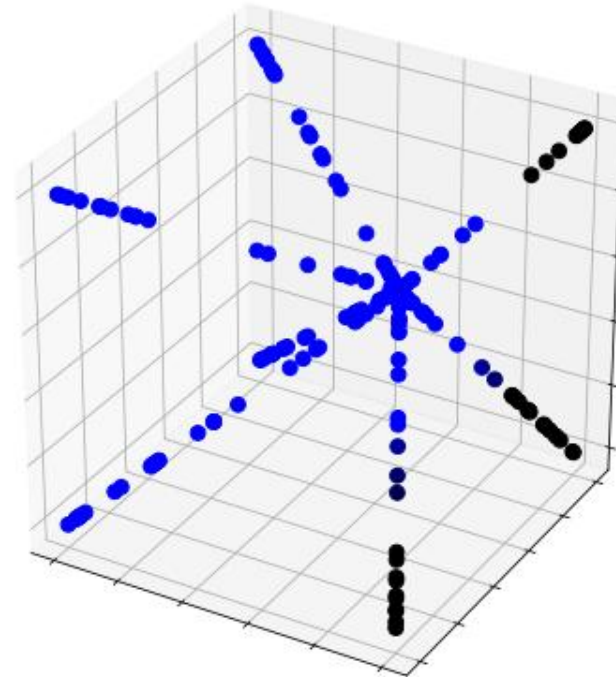
Results

ϵ -Consistent Mixup: Methodology

- Idea: Create simple interpolative structure between observations, but maintain label consistency within ϵ of each observation, and have the model learn the size of this neighborhood.



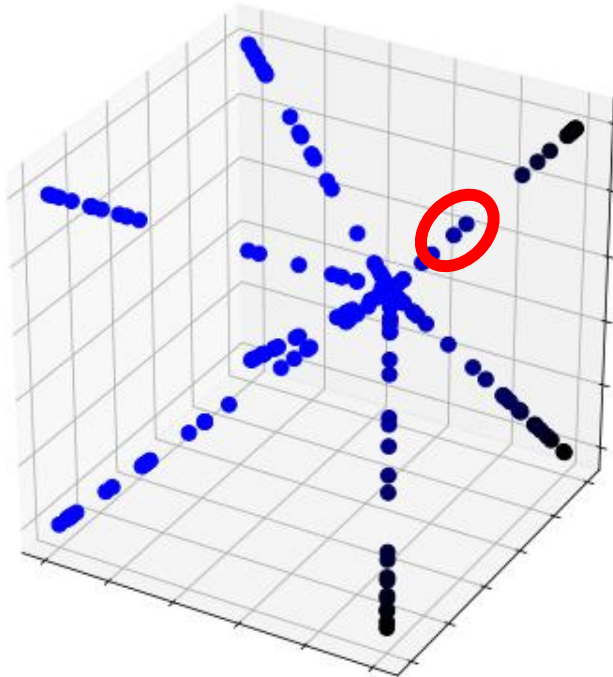
Mixup



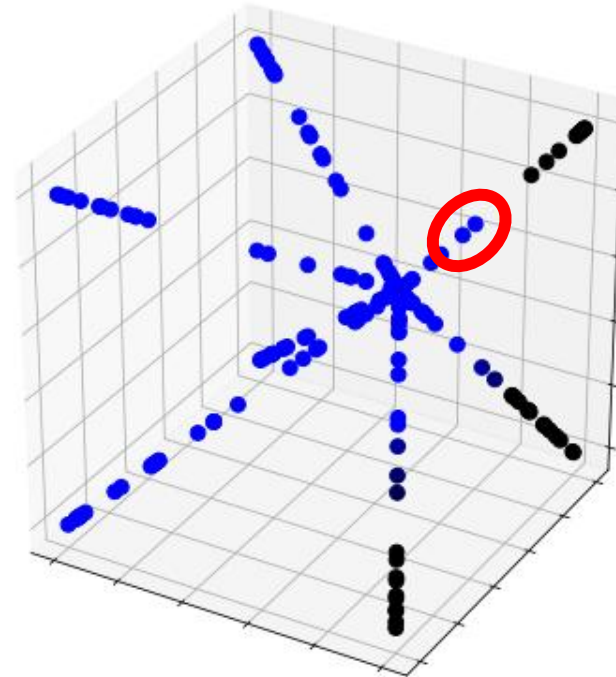
ϵ mu

ϵ -Consistent Mixup: Methodology

- Idea: Create simple interpolative structure between observations, but maintain label consistency within ϵ of each observation, and have the model learn the size of this neighborhood.



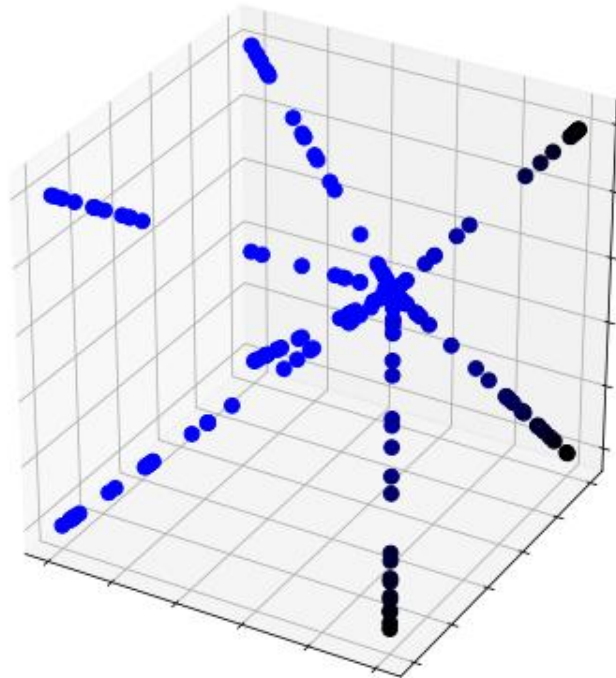
Mixup



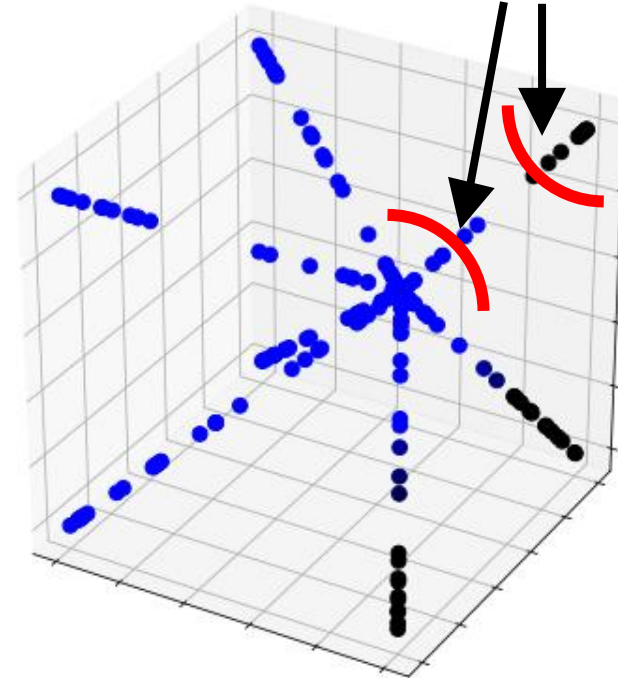
ϵ mu

ϵ -Consistent Mixup: Methodology

- Idea: Create simple interpolative structure between observations, but maintain label consistency within ϵ of each observation, and have the model learn the size of this neighborhood.



Mixup

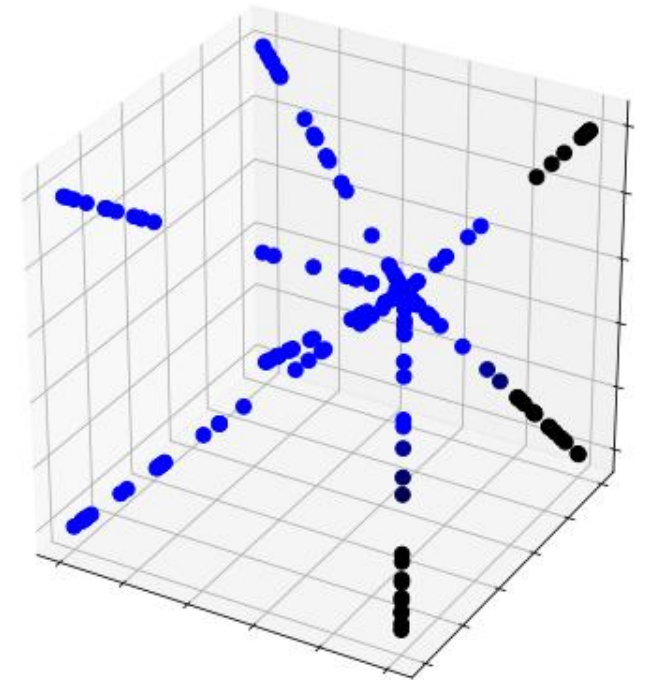


Learned consistency neighborhoods

ϵ mu

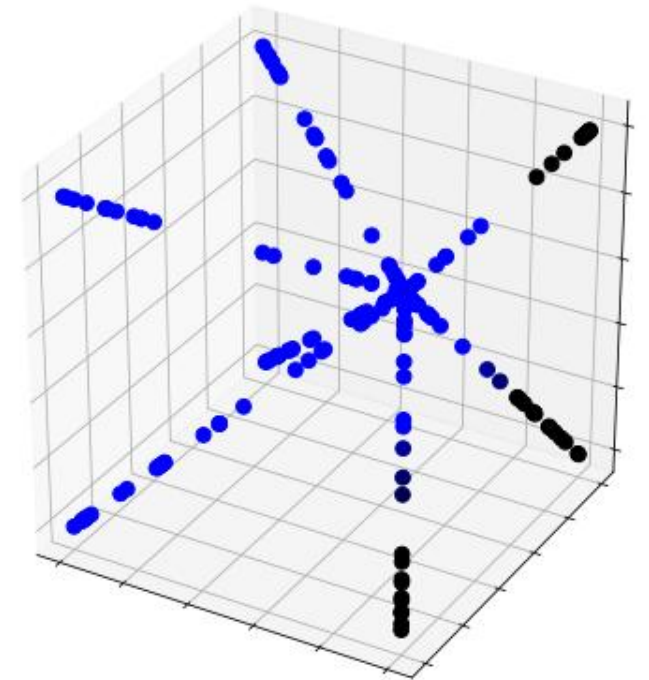
ϵ -Consistent Mixup: Methodology

- Idea: Create simple interpolative structure between observations, but maintain label consistency within ϵ of each observation, and have the model learn the size of this neighborhood.
- Transformation:
 - $T_{\mathcal{X}}(x_i, x_j) = \lambda x_i + (1 - \lambda)x_j, \lambda \sim \text{Beta}(\beta, \beta)$
 - $T_{\mathcal{Y}}(y_i, y_j) = \eta_{\epsilon}(\lambda)p(y_i) + (1 - \eta_{\epsilon}(\lambda))p(x_j)$
 - $p(y) = \begin{cases} y, y \neq \emptyset \\ \hat{p}, y = \emptyset \end{cases}$
- Loss: MSE, KL-divergence, cross-entropy, etc.



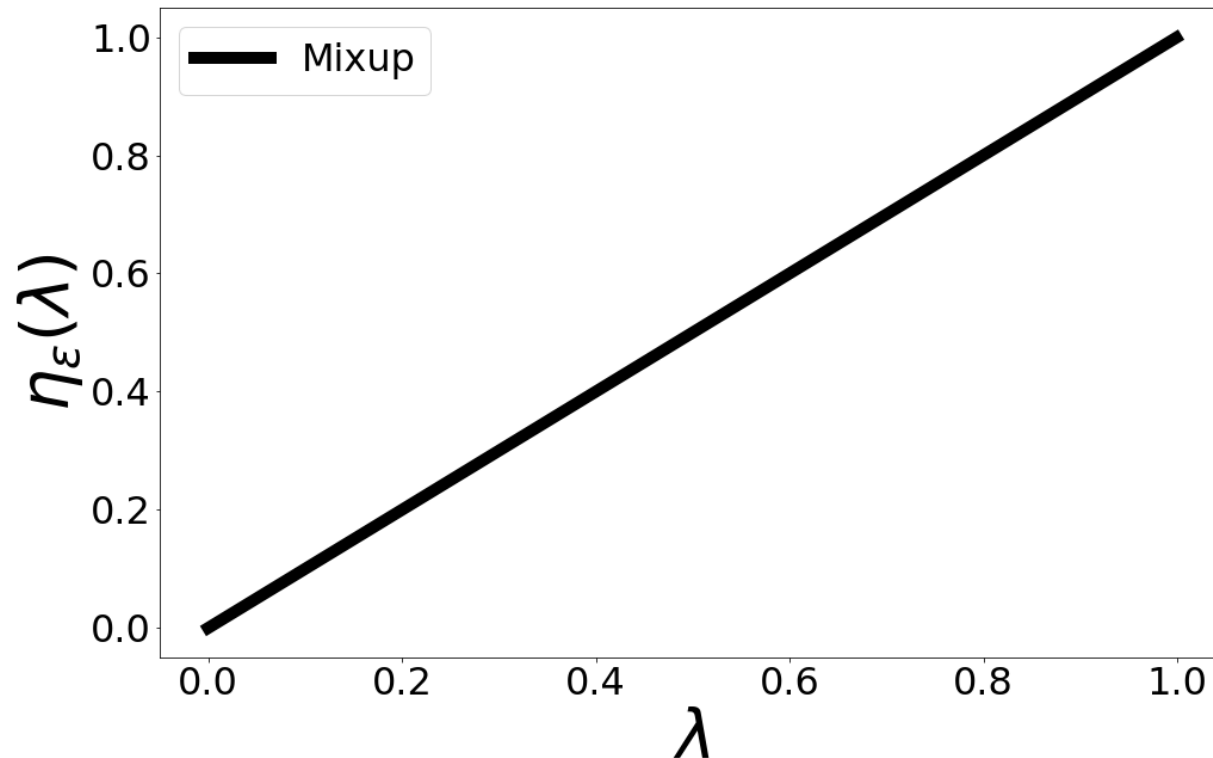
ϵ -Consistent Mixup: Methodology

- Idea: Create simple interpolative structure between observations, but maintain label consistency within ϵ of each observation, and have the model learn the size of this neighborhood.
- Transformation:
 - $T_{\mathcal{X}}(x_i, x_j) = \lambda x_i + (1 - \lambda)x_j, \lambda \sim \text{Beta}(\beta, \beta)$
 - $T_{\mathcal{Y}}(y_i, y_j) = \eta_{\epsilon}(\lambda)p(y_i) + (1 - \eta_{\epsilon}(\lambda))p(x_j)$
 - $p(y) = \begin{cases} y, y \neq \emptyset \\ \hat{p}, y = \emptyset \end{cases}$
- Loss: MSE, KL-divergence, cross-entropy, etc.



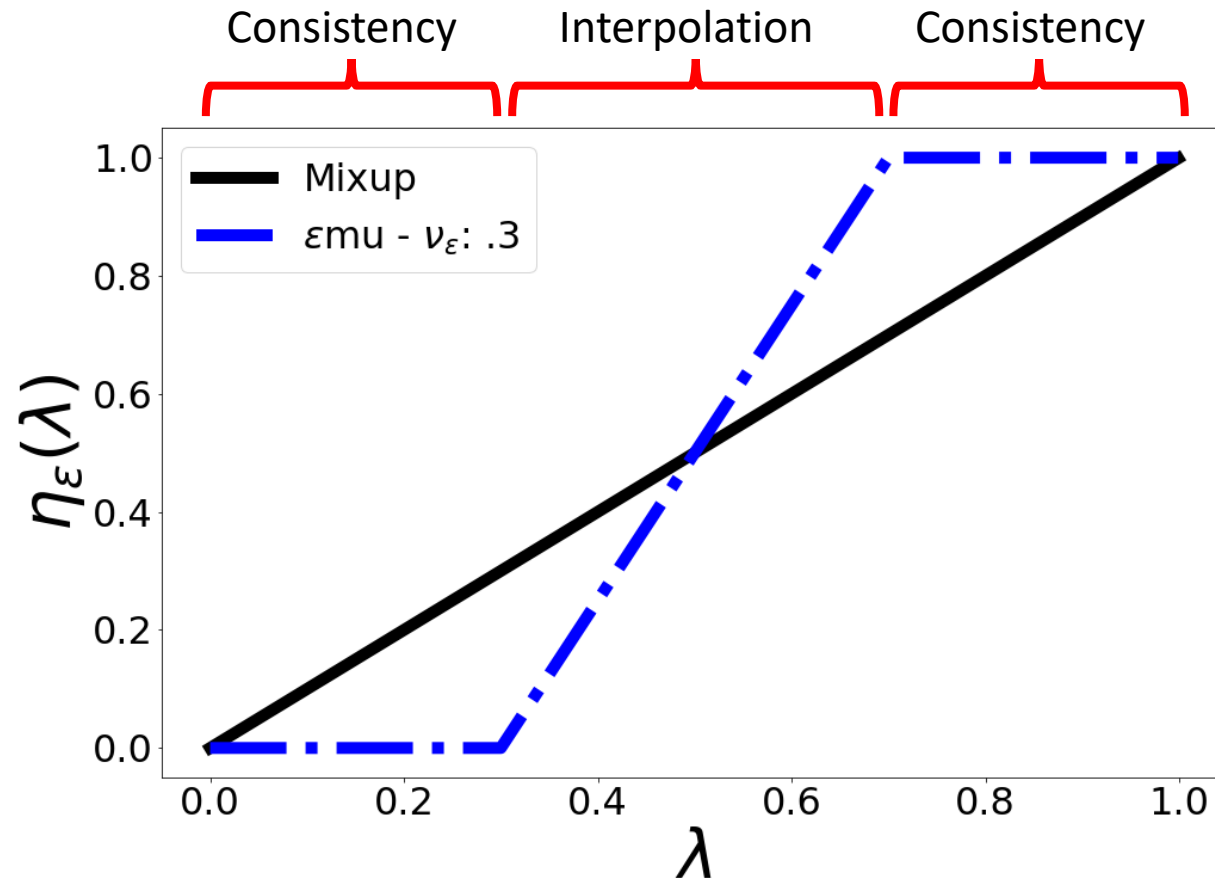
ϵ -Consistent Mixup: Methodology

$$T_y(y_i, y_j) = \eta_\epsilon(\lambda)p(y_i) + (1 - \eta_\epsilon(\lambda))p(x_j)$$



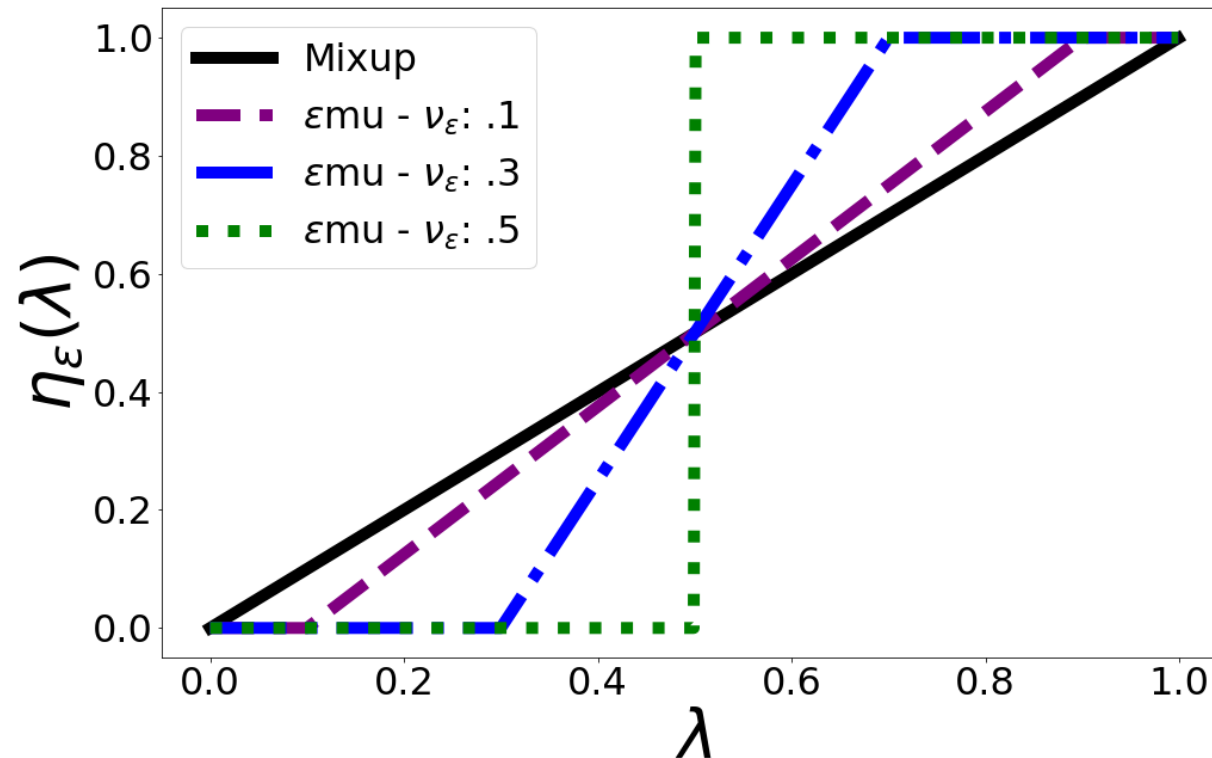
ϵ -Consistent Mixup: Methodology

$$T_y(y_i, y_j) = \eta_\epsilon(\lambda)p(y_i) + (1 - \eta_\epsilon(\lambda))p(x_j)$$



ϵ -Consistent Mixup: Methodology

$$T_y(y_i, y_j) = \eta_\epsilon(\lambda)p(y_i) + (1 - \eta_\epsilon(\lambda))p(x_j)$$



ϵ -Consistent Mixup: Methodology

Algorithm 1: ϵ -Consistent Mixup (ϵ mu) Training

input : A dataset $D^N = (X^N, Y^N)$, neural network classifiers of a given architecture
 $\mathcal{F} = \{f_\theta : \theta \in \Theta\}$, a stochastic interpolation transformation function
 $T = (T_{\mathcal{X}}, T_{\mathcal{Y}})$, hyperparameter β , a structural loss L_S , a structural loss weight w_S , and the supervised cross-entropy loss L_{CE}

output: The loss minimizing classifier $f_{\hat{\theta}}$

- 1 **for** $i = 1$ **to** I **do**
- 2 Generate a batch of size n from D^N : $d_i^n = (X_i^n, Y_i^n)$
- 3 Generate synthetic batch: $\tilde{d}_i^n = (\tilde{X}_i^n, \tilde{Y}_i^n)$
- 4 Randomly pair observations and interpolate by $\lambda \sim \text{Beta}(\beta, \beta)$:
- 5 $t_{\mathcal{X}}(x_j, x_k) = \lambda x_j + (1 - \lambda)x_k$
- 6 $t_{\mathcal{Y}}(y_j, y_k) = \eta_\epsilon(\lambda)p(y_j) + (1 - \eta_\epsilon(\lambda))p(y_k)$
- 7 Specify the total loss: $L(f) = L_{CE}(d_i^n, f) + w_S L_S(\tilde{d}_i^n, f)$
- 8 Compute gradient of total loss w.r.t (θ, ϵ) : $g_{(\theta, \epsilon)} = \nabla_{(\theta, \epsilon)} L$
- 9 Update (θ, ϵ) using Adam optimizer
- 10 **end**
- 11 **return** $f_{\hat{\theta}} = f_\theta$

ϵ -Consistent Mixup: Methodology

Algorithm 1: ϵ -Consistent Mixup (ϵ mu) Training

input : A dataset $D^N = (X^N, Y^N)$, neural network classifiers of a given architecture
 $\mathcal{F} = \{f_\theta : \theta \in \Theta\}$, a stochastic interpolation transformation function
 $T = (T_{\mathcal{X}}, T_{\mathcal{Y}})$, hyperparameter β , a structural loss L_S , a structural loss weight w_S , and the supervised cross-entropy loss L_{CE}

output: The loss minimizing classifier $f_{\hat{\theta}}$

```
1 for  $i = 1$  to  $I$  do
2   Generate a batch of size  $n$  from  $D^N$ :  $d_i^n = (X_i^n, Y_i^n)$ 
3   Generate synthetic batch:  $\tilde{d}_i^n = (\tilde{X}_i^n, \tilde{Y}_i^n)$ 
4   Randomly pair observations and interpolate by  $\lambda \sim \text{Beta}(\beta, \beta)$ :
5      $t_{\mathcal{X}}(x_j, x_k) = \lambda x_j + (1 - \lambda)x_k$ 
6      $t_{\mathcal{Y}}(y_j, y_k) = \eta_\epsilon(\lambda)p(y_j) + (1 - \eta_\epsilon(\lambda))p(y_k)$ 
7   Specify the total loss:  $L(f) = L_{CE}(d_i^n, f) + w_S L_S(\tilde{d}_i^n, f)$ 
8   Compute gradient of total loss w.r.t  $(\theta, \epsilon)$ :  $g_{(\theta, \epsilon)} = \nabla_{(\theta, \epsilon)} L$ 
9   Update  $(\theta, \epsilon)$  using Adam optimizer
10 end
11 return  $f_{\hat{\theta}} = f_\theta$ 
```

Batching

Iterative optimization

ϵ -Consistent Mixup: Results

- Outperforms consistency regularization methods (Π -Model, Mean Teacher, VAT)
- Outperforms comparable Mixup-based method (ICT)
- Competitive with multi-method algorithms (MixMatch, UDA)

CIFAR10 Misclassification Error Rates			SVHN Misclassification Error Rates		
Method	40 Labels	250 Labels	Method	40 Labels	250 Labels
Π -Model	-	53.02 ± 2.05	Π -Model	-	17.65 ± 0.27
Mean Teacher	-	47.32 ± 4.71	Mean Teacher	-	6.45 ± 2.43
VAT	-	36.03 ± 2.82	VAT	-	8.41 ± 1.01
ICT	47.77 ± 2.23	16.76 ± 0.51	ICT	25.52 ± 5.66	5.00 ± 0.28
ϵ mu	37.17 ± 1.87	12.56 ± 1.42	ϵ mu	24.24 ± 4.46	4.37 ± 0.19
MixMatch	47.54 ± 11.50	11.08 ± 0.87	MixMatch	42.55 ± 14.53	3.78 ± 0.26
UDA	29.05 ± 5.93	5.43 ± 0.96	UDA	52.63 ± 20.51	2.72 ± 0.40

ϵ -Consistent Mixup: Results

- Outperforms consistency regularization methods (Π -Model, Mean Teacher, VAT)
- Outperforms comparable Mixup-based method (ICT)
- Competitive with multi-method algorithms (MixMatch, UDA)

CIFAR10 Misclassification Error Rates			SVHN Misclassification Error Rates		
Method	40 Labels	250 Labels	Method	40 Labels	250 Labels
Π -Model	-	53.02 ± 2.05	Π -Model	-	17.65 ± 0.27
Mean Teacher	-	47.32 ± 4.71	Mean Teacher	-	6.45 ± 2.43
VAT	-	36.03 ± 2.82	VAT	-	8.41 ± 1.01
ICT	47.77 ± 2.23	16.76 ± 0.51	ICT	25.52 ± 5.66	5.00 ± 0.28
ϵ mu	37.17 ± 1.87	12.56 ± 1.42	ϵ mu	24.24 ± 4.46	4.37 ± 0.19
MixMatch	47.54 ± 11.50	11.08 ± 0.87	MixMatch	42.55 ± 14.53	3.78 ± 0.26
UDA	29.05 ± 5.93	5.43 ± 0.96	UDA	52.63 ± 20.51	2.72 ± 0.40

ϵ -Consistent Mixup: Results

- Outperforms consistency regularization methods (Π -Model, Mean Teacher, VAT)
- Outperforms comparable Mixup-based method (ICT)
- Competitive with multi-method algorithms (MixMatch, UDA)

CIFAR10 Misclassification Error Rates			SVHN Misclassification Error Rates		
Method	40 Labels	250 Labels	Method	40 Labels	250 Labels
Π -Model	-	53.02 ± 2.05	Π -Model	-	17.65 ± 0.27
Mean Teacher	-	47.32 ± 4.71	Mean Teacher	-	6.45 ± 2.43
VAT	-	36.03 ± 2.82	VAT	-	8.41 ± 1.01
ICT	47.77 ± 2.23	16.76 ± 0.51	ICT	25.52 ± 5.66	5.00 ± 0.28
ϵ mu	37.17 ± 1.87	12.56 ± 1.42	ϵ mu	24.24 ± 4.46	4.37 ± 0.19
MixMatch	47.54 ± 11.50	11.08 ± 0.87	MixMatch	42.55 ± 14.53	3.78 ± 0.26
UDA	29.05 ± 5.93	5.43 ± 0.96	UDA	52.63 ± 20.51	2.72 ± 0.40

ϵ -Consistent Mixup: Results

- Outperforms consistency regularization methods (Π -Model, Mean Teacher, VAT)
- Outperforms comparable Mixup-based method (ICT)
- Competitive with multi-method algorithms (MixMatch, UDA)

CIFAR10 Misclassification Error Rates			SVHN Misclassification Error Rates		
Method	40 Labels	250 Labels	Method	40 Labels	250 Labels
Π -Model	-	53.02 ± 2.05	Π -Model	-	17.65 ± 0.27
Mean Teacher	-	47.32 ± 4.71	Mean Teacher	-	6.45 ± 2.43
VAT	-	36.03 ± 2.82	VAT	-	8.41 ± 1.01
ICT	47.77 ± 2.23	16.76 ± 0.51	ICT	25.52 ± 5.66	5.00 ± 0.28
ϵ mu	37.17 ± 1.87	12.56 ± 1.42	ϵ mu	24.24 ± 4.46	4.37 ± 0.19
MixMatch	47.54 ± 11.50	11.08 ± 0.87	MixMatch	42.55 ± 14.53	3.78 ± 0.26
UDA	29.05 ± 5.93	5.43 ± 0.96	UDA	52.63 ± 20.51	2.72 ± 0.40

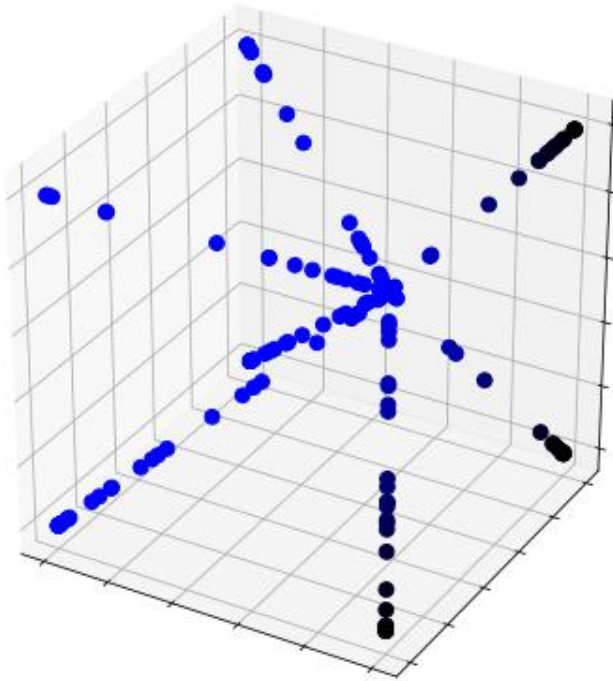
ϵ -Consistent Mixup: Analysis

- ϵ mu produces higher quality synthetic labels $\tilde{y}$ than Mixup and establishes its structure across a larger region of the feature space

ϵ -Consistent Mixup: Analysis

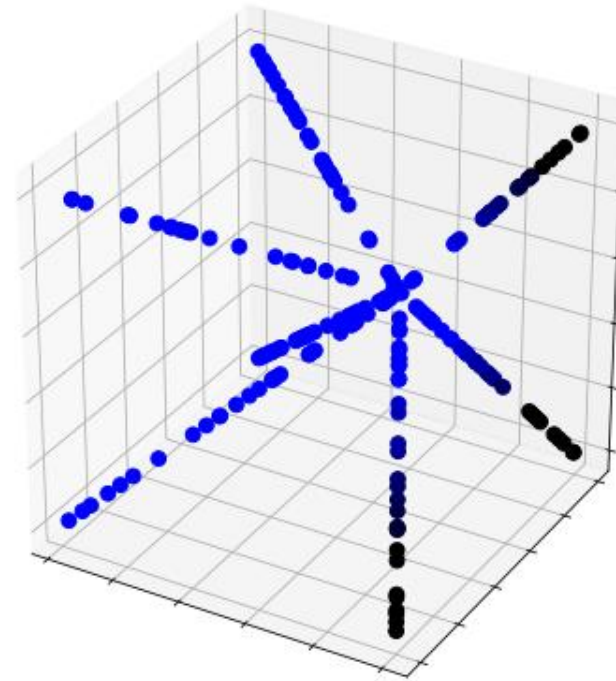
- ϵ mu produces higher quality synthetic labels $\tilde{y}$ than Mixup and establishes its structure across a larger region of the feature space

Mixup - Optimal Setting



$\lambda \sim \text{Beta}(0.2, 0.2)$

ϵ mu - Optimal Setting



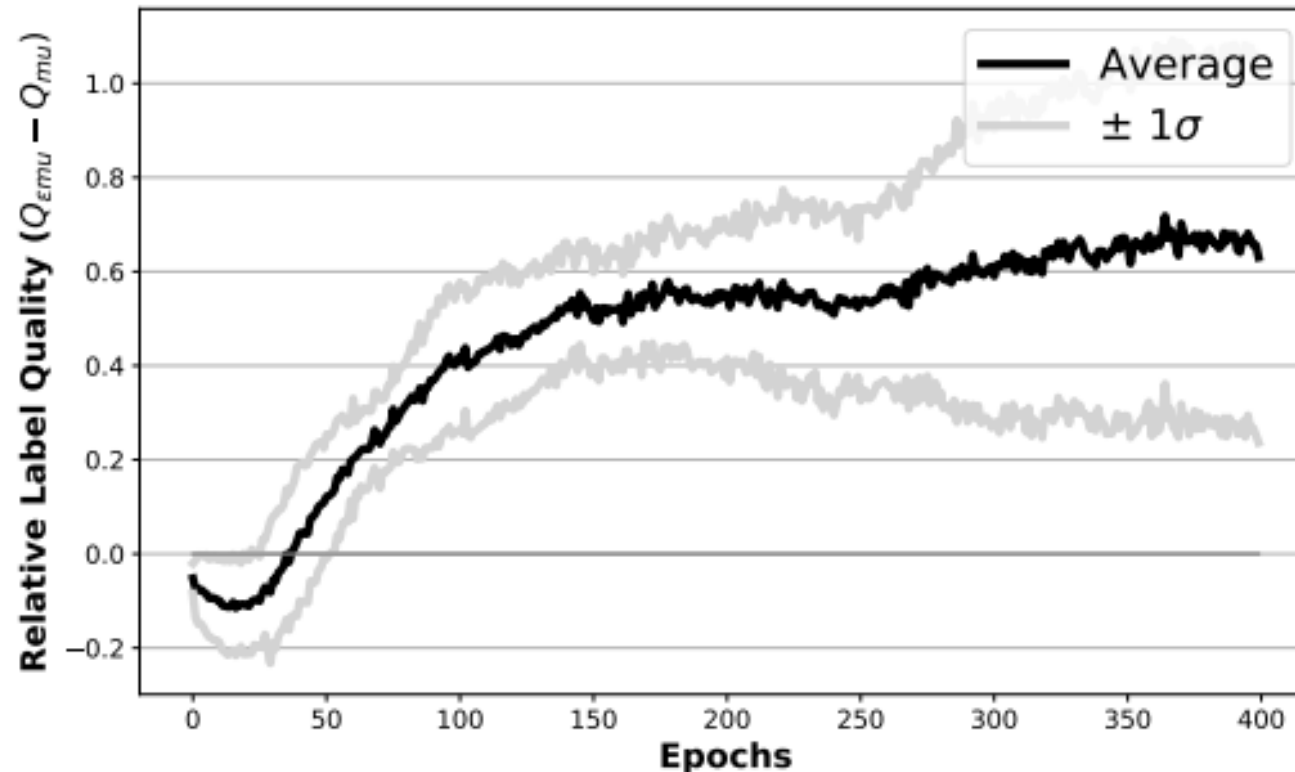
$\lambda \sim \text{Beta}(1.0, 1.0)$

ϵ -Consistent Mixup: Analysis

- ϵ mu produces higher quality synthetic labels $\tilde{y}$ than Mixup and establishes its structure across a larger region of the feature space
- Quality: $Q(\tilde{y}_i) = \frac{1}{D(\tilde{y}_i, y_i^*)}$
- $y_i^* = f_{\theta^*}(\tilde{x}_i)$, where f_{θ^*} is a classifier trained on the full supervised dataset (a substitute for the unknown true label y_i)
- Relative Label Quality: $Q(\tilde{y}_i^{\epsilon mu}) - Q(\tilde{y}_i^{Mixup})$

ϵ -Consistent Mixup: Analysis

- ϵ mu produces higher quality synthetic labels $\tilde{y}$ than Mixup and establishes its structure across a larger region of the feature space



ϵ -Consistent Mixup: Analysis

- ϵ mu-trained models produces more confident predictions than models trained with Mixup

ϵ -Consistent Mixup: Analysis

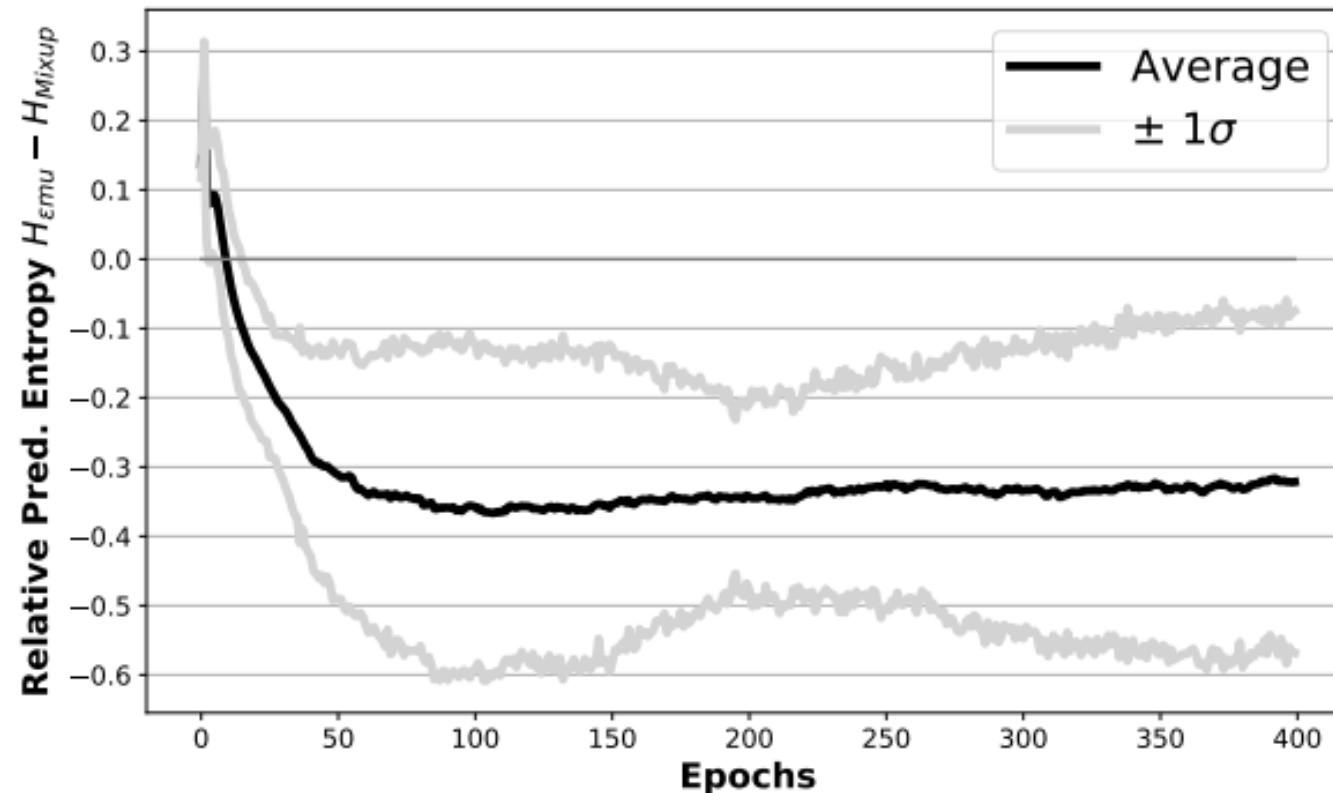
- ϵ mu-trained models produces more confident predictions than models trained with Mixup
- Prediction Uncertainty (Shannon Entropy):

$$H(f_{\theta}(\tilde{x}_i)) = - \sum_{c=1}^C f_{\theta}(\tilde{x}_i)_c \log(f_{\theta}(\tilde{x}_i)_c)$$

- Relative Prediction Entropy:
 - $H(f_{\theta^{\epsilon mu}}(\tilde{x}_i)) - H(f_{\theta^{Mixup}}(\tilde{x}_i))$

ϵ -Consistent Mixup: Analysis

- ϵ mu-trained models produces more confident predictions than models trained with Mixup

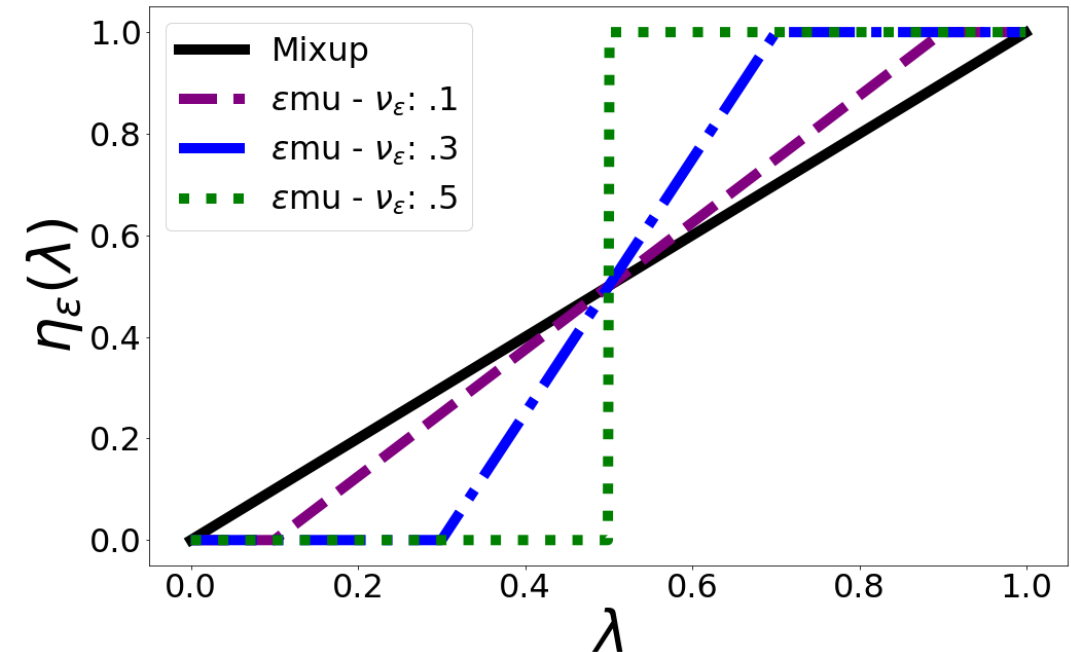


Appendix

ϵ -Consistent Mixup

$$T_y(y_i, y_j) = \eta_\epsilon(\lambda)p(y_i) + (1 - \eta_\epsilon(\lambda))p(x_j)$$

- $t_x(x_i, x_j) = \lambda x_i + (1 - \lambda)x_j, \lambda \sim \text{Beta}(\beta, \beta)$
- $t_y(y_i, y_j) = \eta_\epsilon(\lambda)p(y_i) + (1 - \eta_\epsilon(\lambda))p(x_j)$
- $$\eta_\epsilon(\lambda) = \begin{cases} 0 & , \lambda \leq v_\epsilon \\ \frac{\lambda - v_\epsilon}{1 - 2v_\epsilon} & , v_\epsilon < \lambda < 1 - v_\epsilon \\ 1 & , \lambda \geq 1 - v_\epsilon \end{cases}$$
- $$v_\epsilon = \frac{\epsilon}{D(x_i, x_j)}$$



Structural Regularization: In General

